

Academic Program Workshop: Microchip PyKit Explorer



A Leading Provider of Smart, Connected and Secure Embedded Control Solutions



SMART | CONNECTED | SECURE

Who Is Microchip Technology?

What Do We Do?



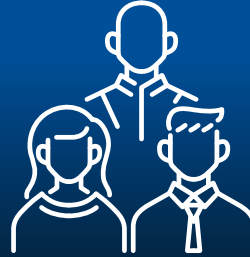
Microchip At a Glance



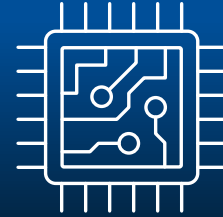
**Founded
February 14, 1989**



**Headquartered in
Chandler, AZ
'The Silicon Desert'**

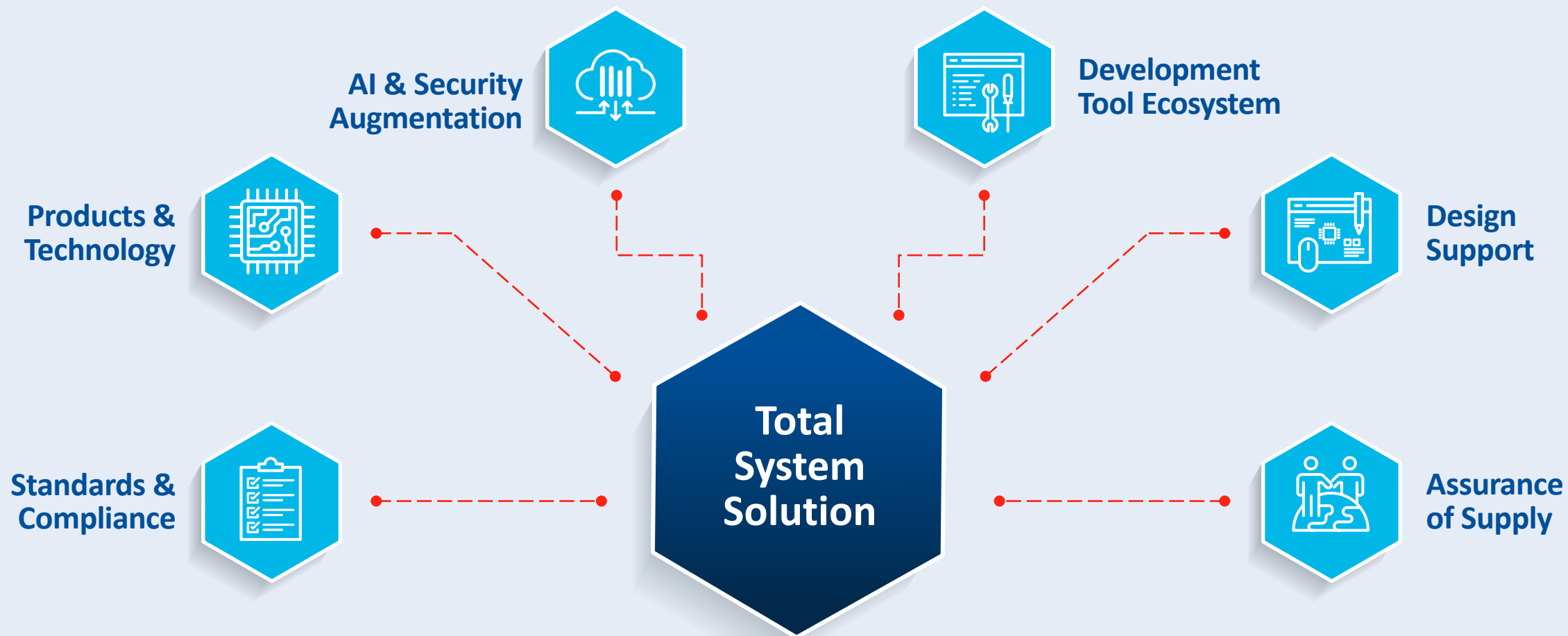


**17,000+
Employees**



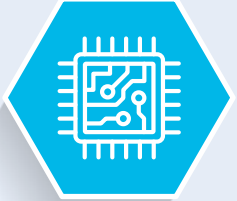
**100,000+
Product Offerings**

Making Innovative Design Easier



An integrated portfolio of products, software, design tools and organizational support to make your design easier.

Our Portfolio



Processors

8-, 16- & 32-Bit MCUs
Digital Signal Controllers
32- & 64-bit Microprocessors
FPGAs PLDs



Connectivity

CAN, CAN2 and CAN FD
Bluetooth, Wi-Fi®, Zigbee
Ethernet, Optical Networking
RF Modules & Microwave Solutions



Touch

MaxTouch® Touchscreen Controllers
GestIC® Air Gesture Controllers
Proximity Touchscreens
MCUs with Capacitive Touch



Sensor & Motor Drive

Current/Voltage/Power Monitor ICS
CO/Smoke Detector Horn Driver ICs
Motor Drivers
Temperature Sensors



Security Products

Authentication ICs
Root of Trust Controllers
MCUs w/ Integrated Security
Secure FPGAs and SoC FPGAs
Trust Platforms



Interface & Connectivity

CAN, CAN2 and CAN FD
Ethernet Switches, PHYs, Controllers
Power-over-Ethernet (PoE)
LIN Bus Networking
Line Circuits and Drivers
PCIe™ Switches
USB Hubs, Switches and Transceivers
USB-C Power Delivery Controllers



Analog & Mixed Signal

In Amps, Op Amps & PGAs
DACs & ADCs
AC/DC Power & Energy Devices
Current/Voltage/Power Monitors
Digital Potentiometers
Voltage References



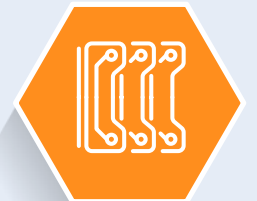
Power Management

Silicon Carbide Products
MOSFETs
LDOs and Battery Management
DC-DC Switching Converters
Voltage Supervisors & References
Battery Chargers Gate Drivers
AC-DC Power Converters
Electromechanical Voltage Relays



Clock & Timing

Atomic Clocks
GNSS Disciplined Oscillators
Clock Generators and Buffers
Oscillators
Real-Time Clocks
Jitter Attenuators
PCIe™ Timers
High Performance Timing Systems



Memory & Storage

Serial EERAM & Serial EEPROM
Serial & Parallel FLASH Memory
Serial SRAM & Serial NVSRAM
Adaptec Host Bus Adaptor
Adaptec SmartRAID RAID Adaptors
Smart IOC I/O Controllers
SXP SAS Expanders

How Many Microchip Parts Are In This Kit?

Total System Solution

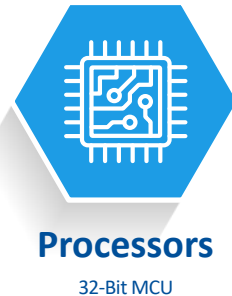
- **Voltage Regulators**

- MCP1826S – 3.3V output
- MIC5309 – 1.8V output



- **Microcontroller**

- ATSAME51 32-bit MCU



- **Flash Memory**

- SST26VF032B – 4MB Flash storage



- **Interface**

- ATA6561 CAN Transceiver



- **Bluetooth Low Energy (BLE)**

- RNBD451 BLE Module



What Is This Workshop About?

Microchip Academic Program



- **Introduction to Embedded Systems Programming**
 - Give students the “Wow, I just built something cool!” moment
- **What is an Embedded System?**
 - Specialized computer system with a dedicated function
 - Combines hardware and software
 - Controls a specific task within a larger mechanical or electronic device
 - Physical Computing
 - Sense the world around it
 - Process the sensor input
 - Control something

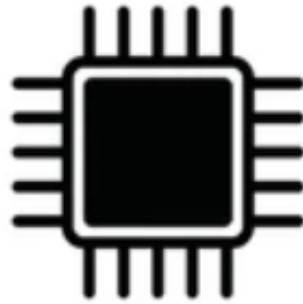
What is CircuitPython?

Python that runs directly on microcontrollers



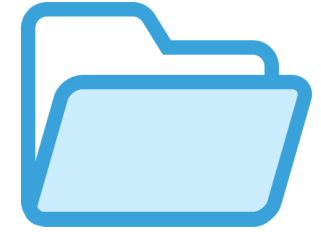
Python Syntax

Write real Python code: variables, loops, methods, and libraries, with no compilation step.



Hardware Access

Read sensors, drive motors, control NeoPixels, and talk to displays in just a few lines of code.



Drag-and-Drop Editing

The board mounts as a USB drive. Edit code.py in any text editor and watch it run instantly

Why CircuitPython?



Beginner Friendly

Python is the world's most-taught language. Students already know the basics.



Fast Iteration

Save the file, see it run. No compile, flash, or reset cycle to slow you down.



Rich Library Support

Drivers for hundreds of sensors, displays, and peripherals, all ready to import.

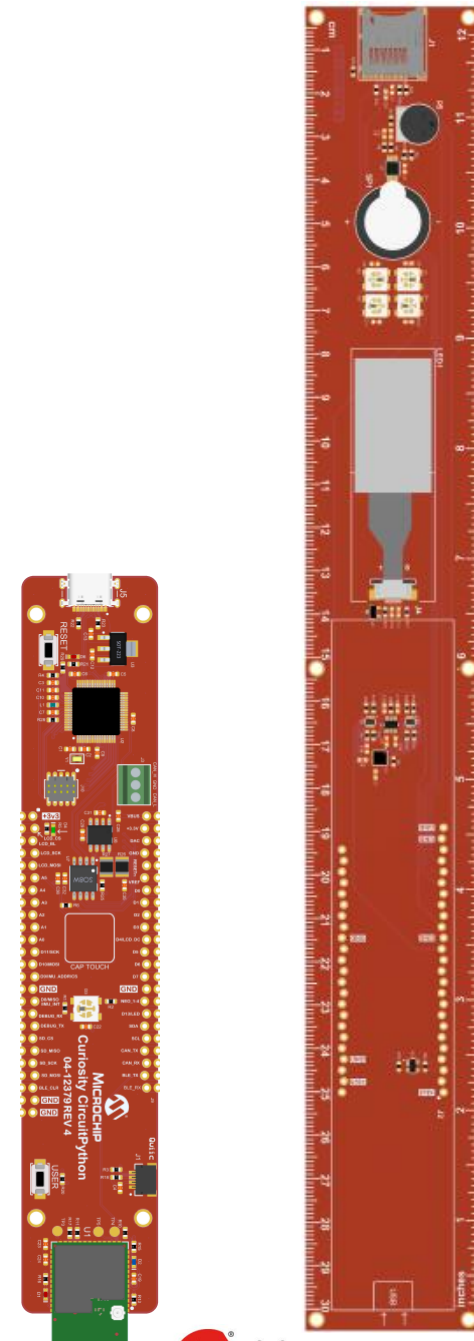
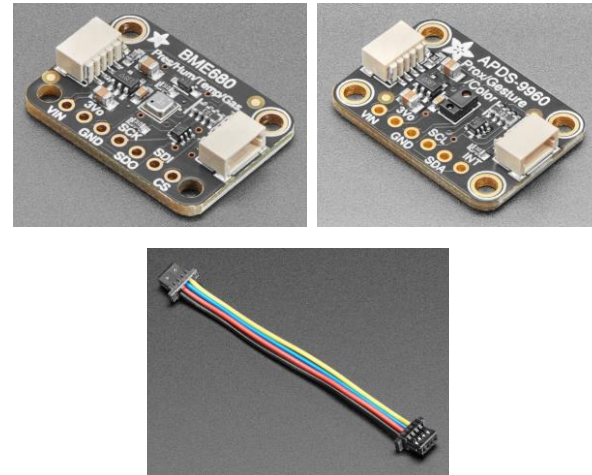


Huge Community

Microchip, Adafruit, Arduino, and Maker communities provide tutorials, forums, and examples.

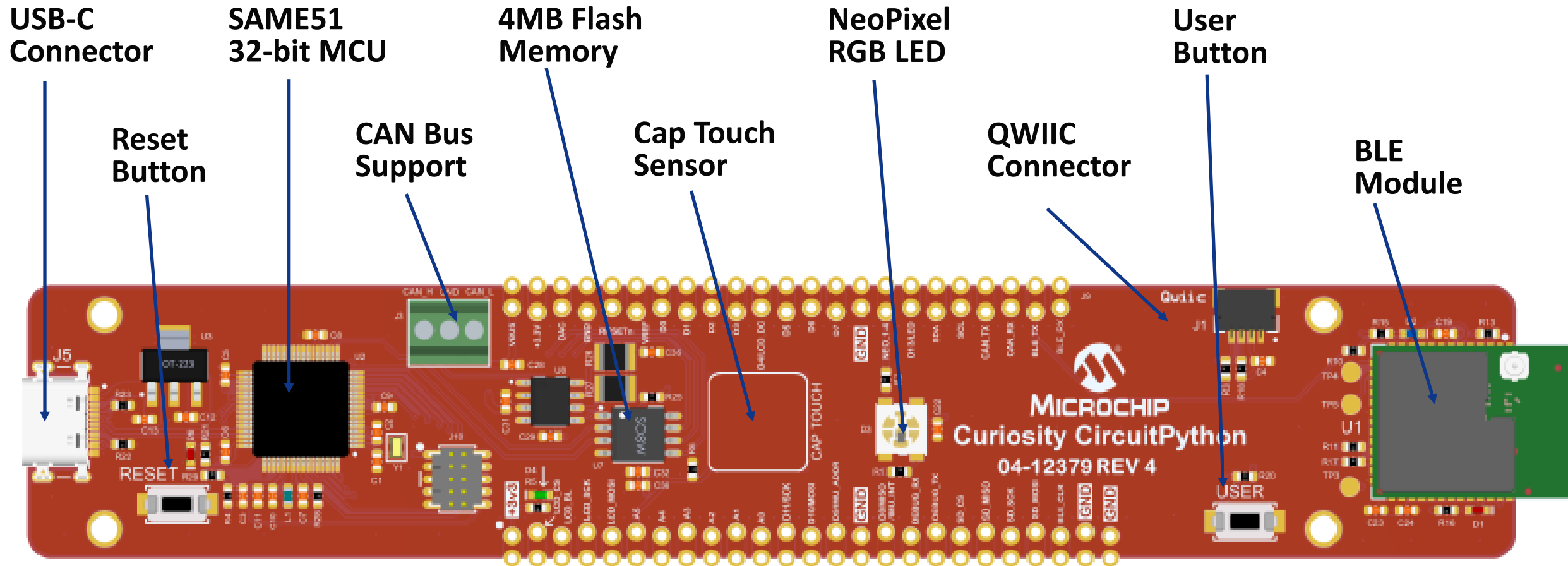
What's In The Kit?

- Curiosity CircuitPython dev board
- PyKit Ruler
- **Adafruit BME680 Breakout:**
 - Temp, Humidity, Pressure, Gas sensor
- **Adafruit APDS9960 Breakout:**
 - Proximity, Gesture, Color sensor
- **QWIIC / STEMMA cable**
- **USB-A to USB-C cable**
- **Header Pins to connect Ruler and dev board**
 - 1 x 23-pin, 1 x 25-pin



What Hardware Does The Kit Have?

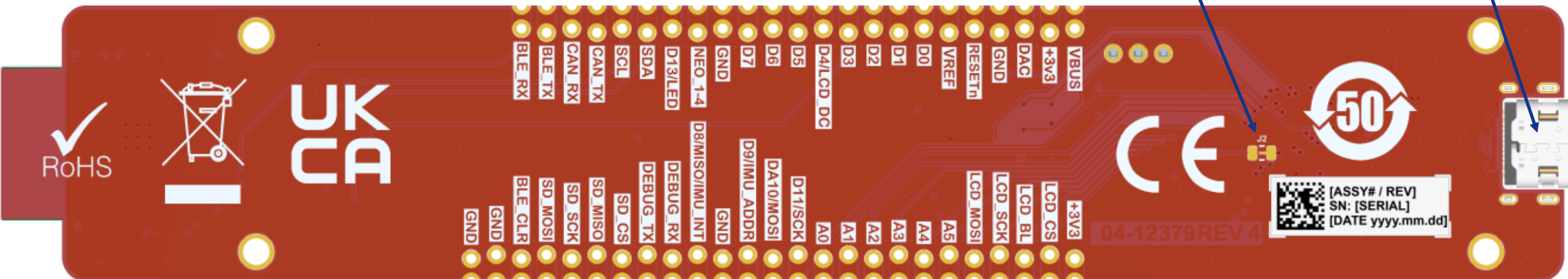
Curiosity CircuitPython dev board



Curiosity CircuitPython dev board

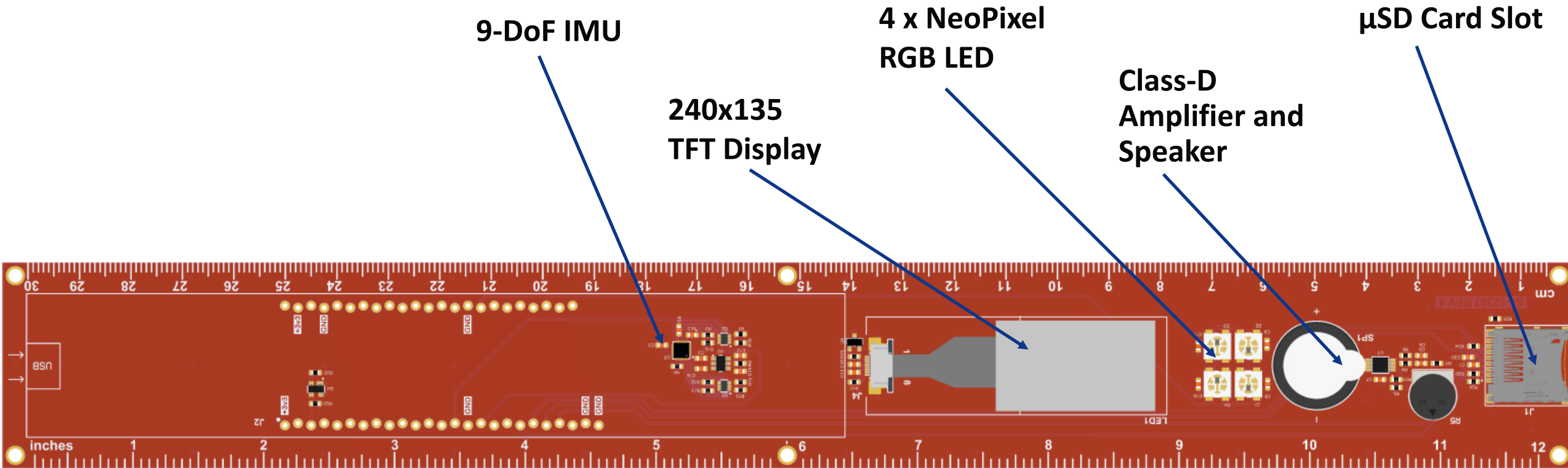
Full Pinout

USB-C Connector



What Hardware Does The Kit Have?

PyKit Ruler baseboard



What Hardware Does The Kit Have?

PyKit Ruler baseboard - backside

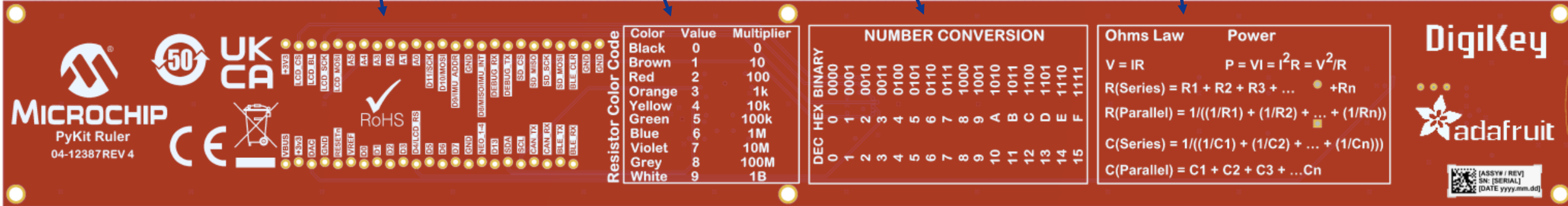
Full Pinout

Resistor
Color Code

Decimal
Binary
Hex
Number
Conversions

Useful Equations:
Ohm's Law
Series/Parallel
Resistors/Capacitors

Our Partners



Do We Need To Solder?

- **No Soldering Required!**
- **Both boards use a friction fit**
 - Easy to put together
 - Easy to take apart
 - Good signal integrity!



Fit header pins to Curiosity CircuitPython dev board

- **Group task:**

1. Fit **short side of header pins** to Curiosity CircuitPython board

Start from one end of the header and work your way down , gently wiggling while pushing pins in

2. Connect Curiosity CircuitPython board to Ruler

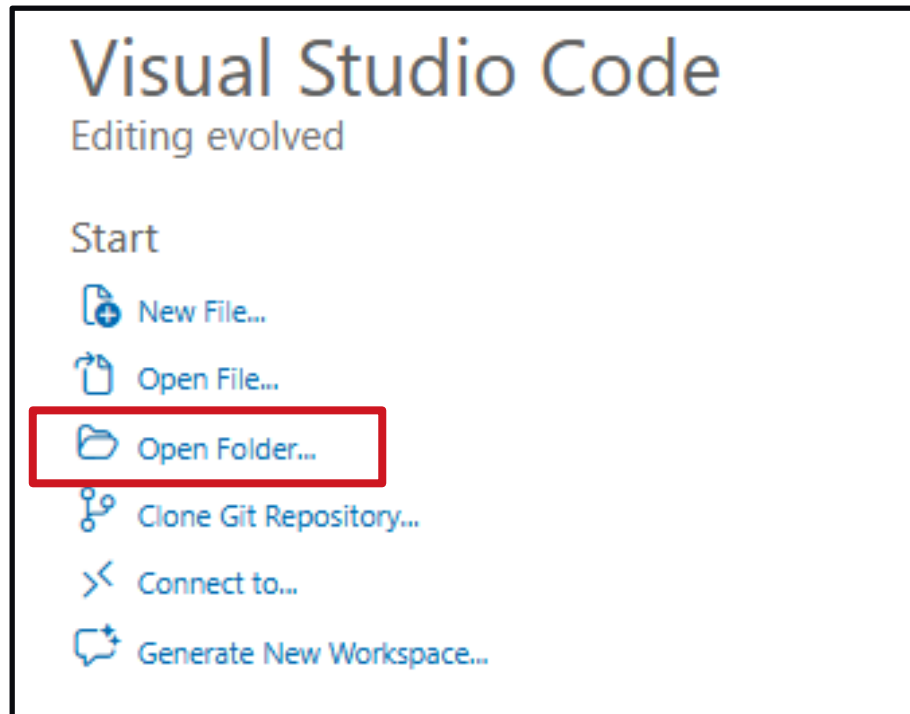
Start from USB end of Curiosity board and work your way down, gently wiggling while pushing pins in



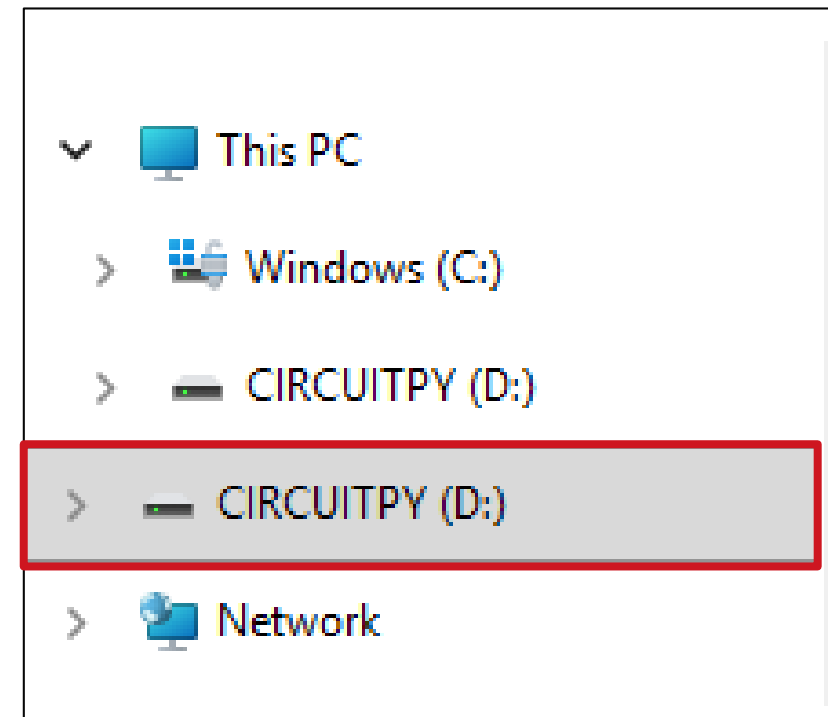
Editing CircuitPython Code in VS Code

Connect CircuitPython kit to your computer with USB cable

1. Open VS Code:

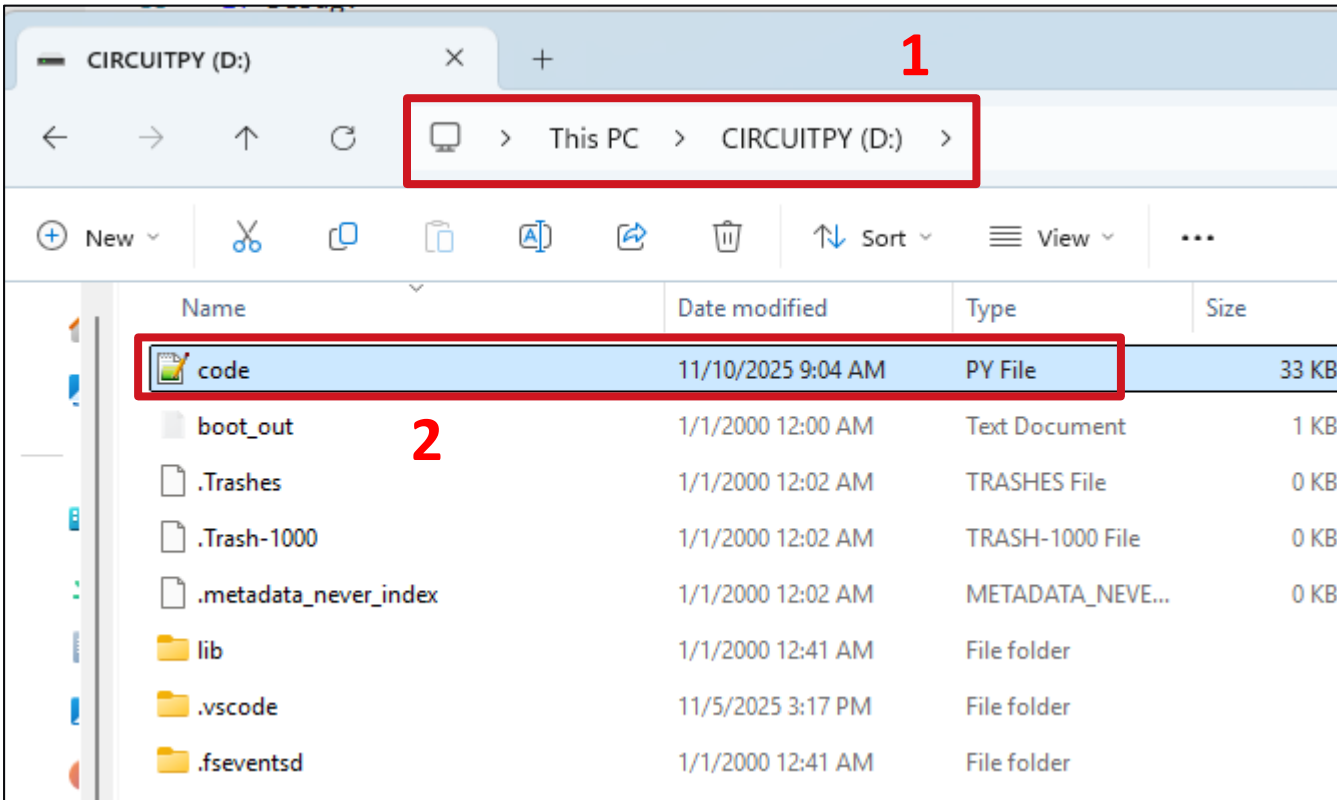


2. Open CIRCUITPY folder:



How to Run Your Code

code.py file

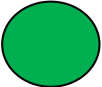
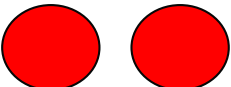


For your code to run on the board, it must:

1. Be in the **root directory** of D:\CIRCUITPY
2. Be named ***code.py***

NeoPixel Status Indicator

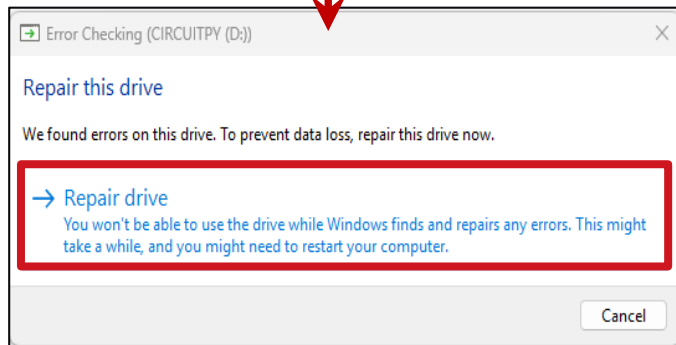
Visual Error Indication

- **NeoPixel - 1 Green Flash** 
 - Code completed successfully
- **NeoPixel - 2 Red Flashes** 
 - Code encountered an error – Check the Serial Monitor for info
- **NeoPixel - 3 Yellow Flashes** 
 - Safe Mode entered after bad crash!
 - Need to preset Reset button to exit Safe Mode
- **Note:** Do NOT confuse NeoPixel flash with BLE module **Blue LED** (advertising mode) flash!

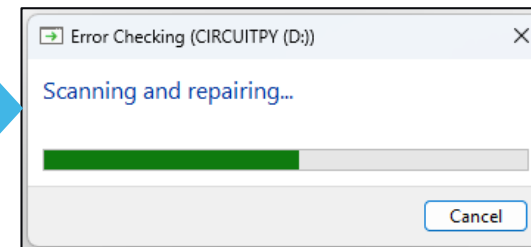
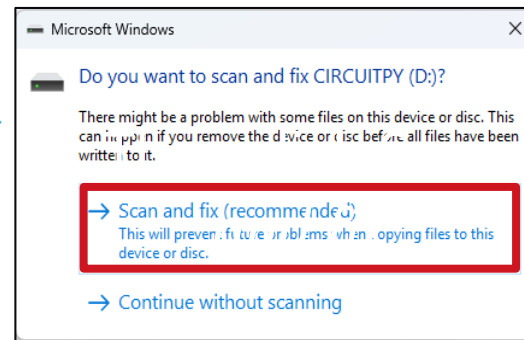
Connect The Curiosity CircuitPython

To your laptop using the included USB cable

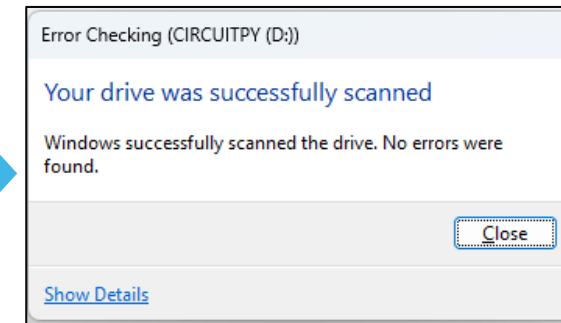
If you see this, click
Repair Drive



Then click
Scan and fix



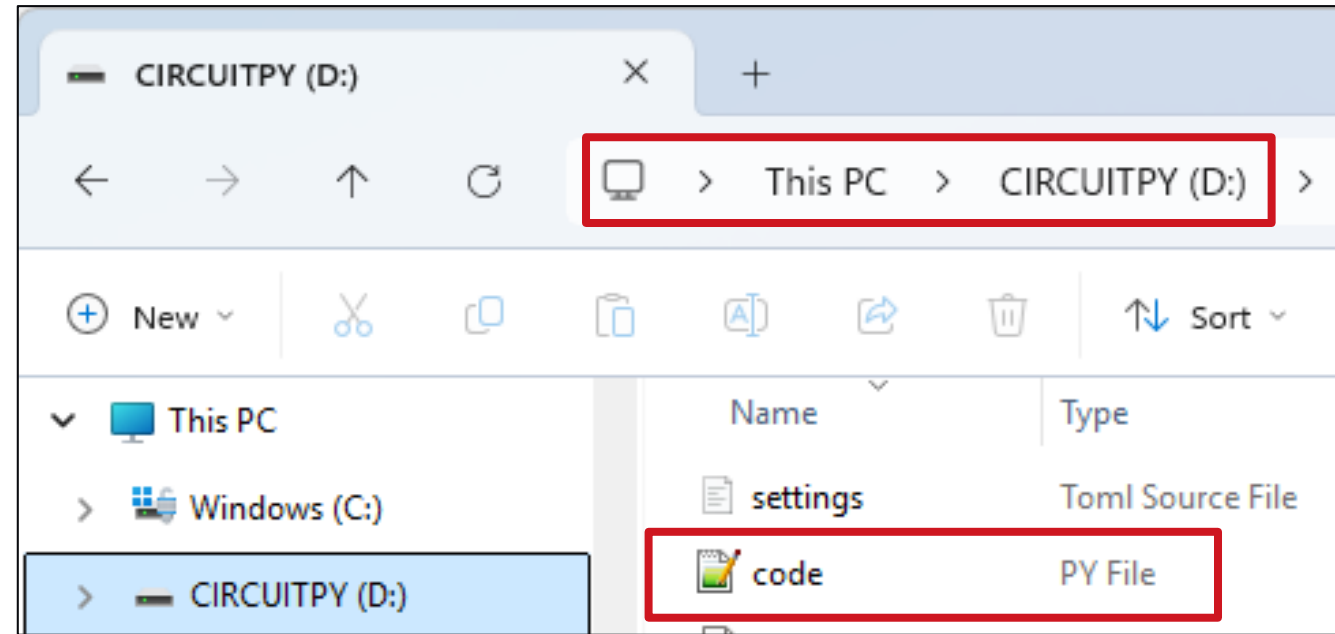
Click ***Close***



How Do I Program The Board?

Just Save The File `code.py`

- Q: Where should you save your `code.py` file??
- A: In the root directory of:
 - D:\CIRCUITPY
- **Note:** Saving `code.py` inside a folder in D:\CIRCUITPY will **NOT** work! It **MUST** be in the **ROOT DIRECTORY**
- On Windows:
 - Make sure you eject drive before pressing reset button or disconnecting!
 - Group task: Eject the drive

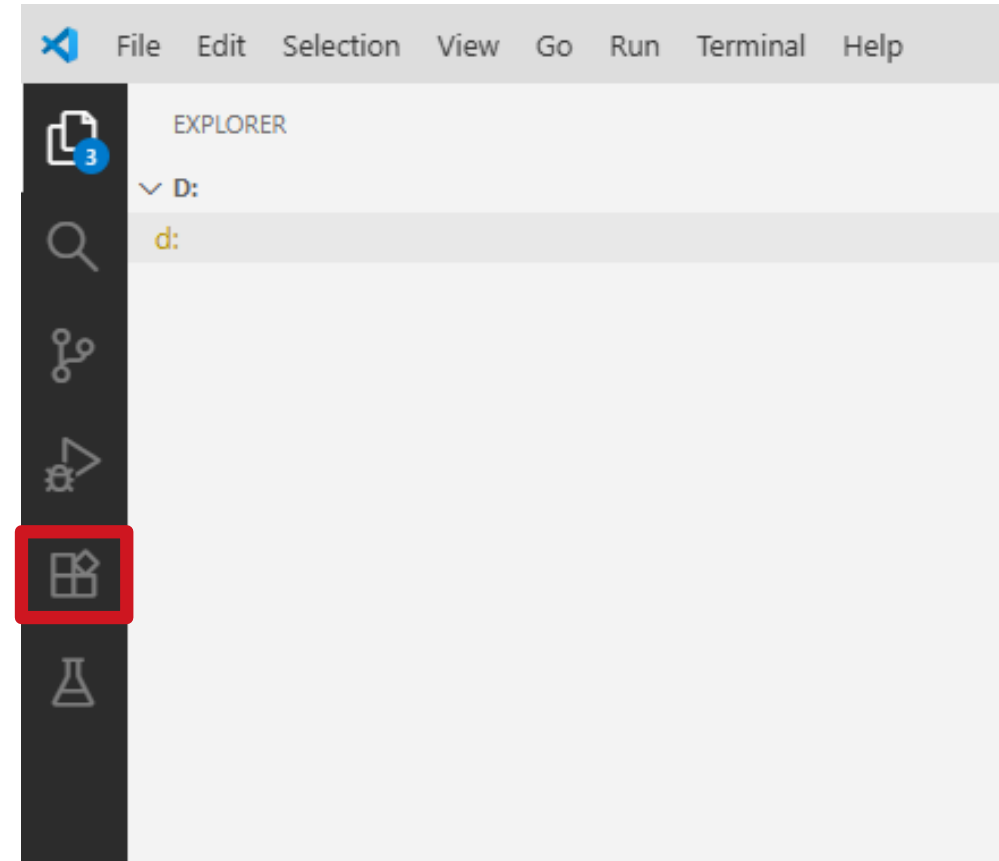
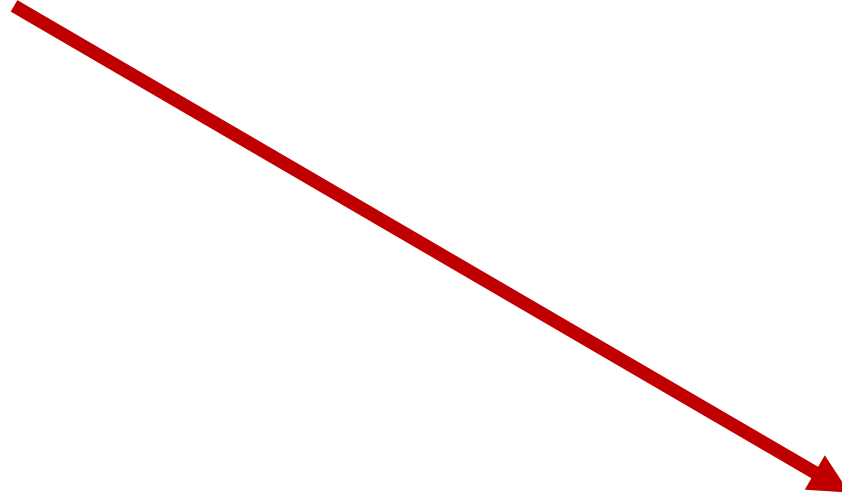


Open Serial Monitor

VS Code Extension



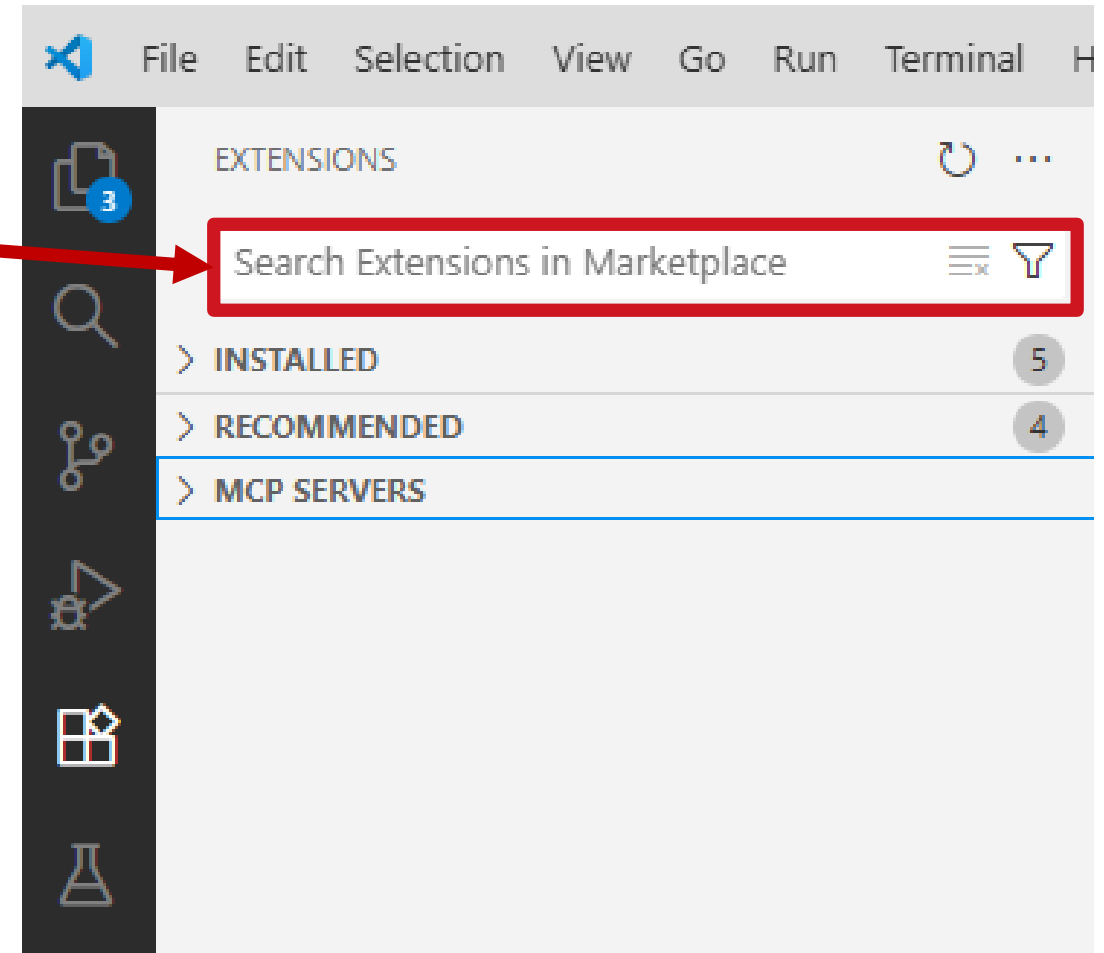
- Open *VS Code*
- Click on *Extensions* icon on left side



Search For Serial Monitor Extension

VS Code Extensions

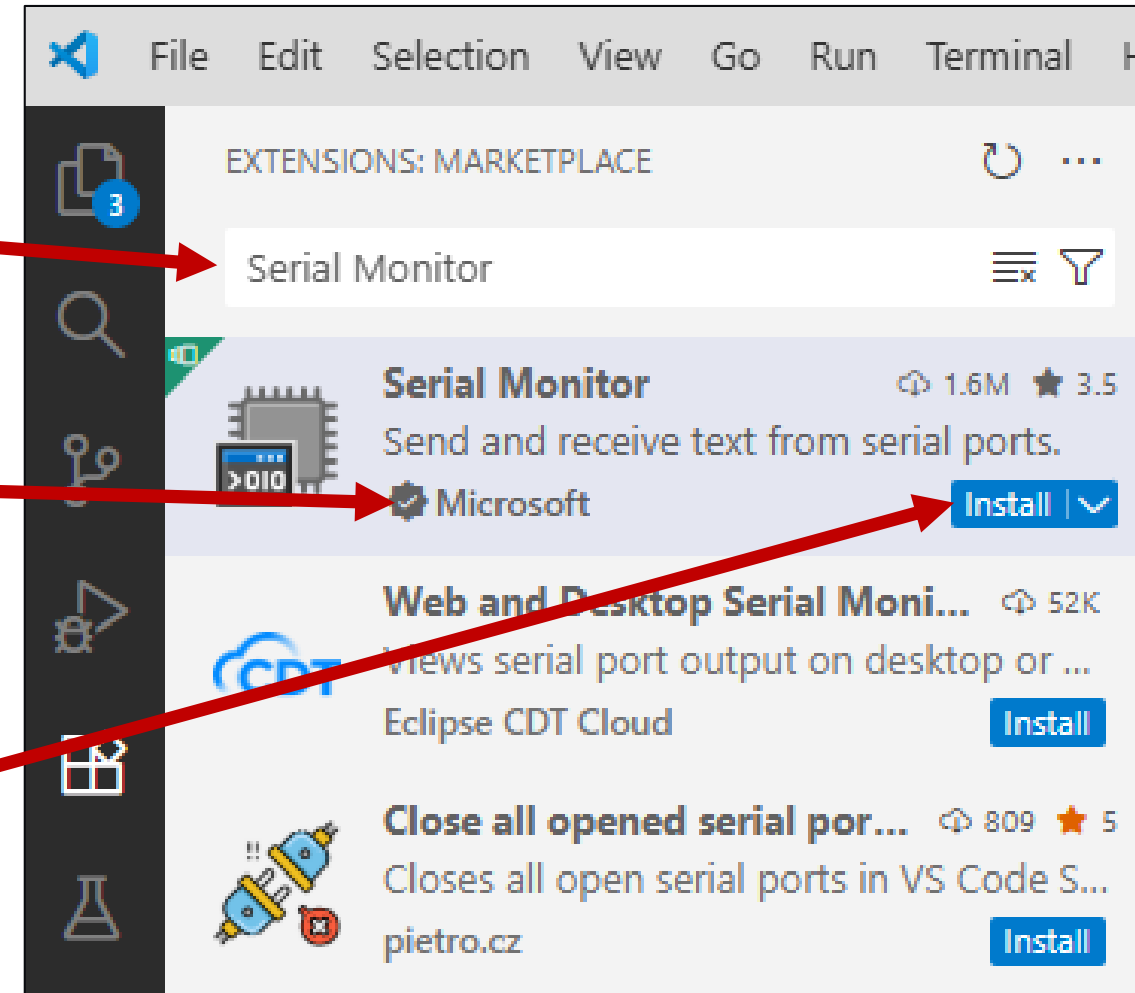
- **Click on text field:**
 - *Search Extensions in Marketplace*



Enter “Serial Monitor” in Text Field

VS Code Extensions

- Type in Text Field:
 - *Serial Monitor*
- Confirm author is *Microsoft*
- Click *Install* button



Open Serial Monitor

VS Code - Toggle Panel

- In the top right corner

- Click *Toggle Panel*



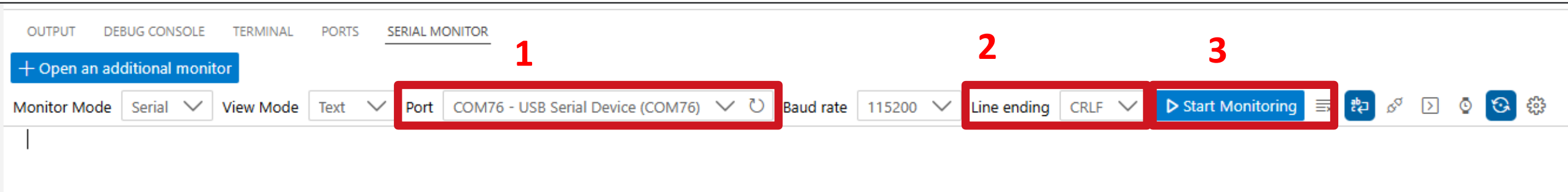
1. Select your COM port:

- *USB Serial Device*

2. Line Ending:

- *CRLF*

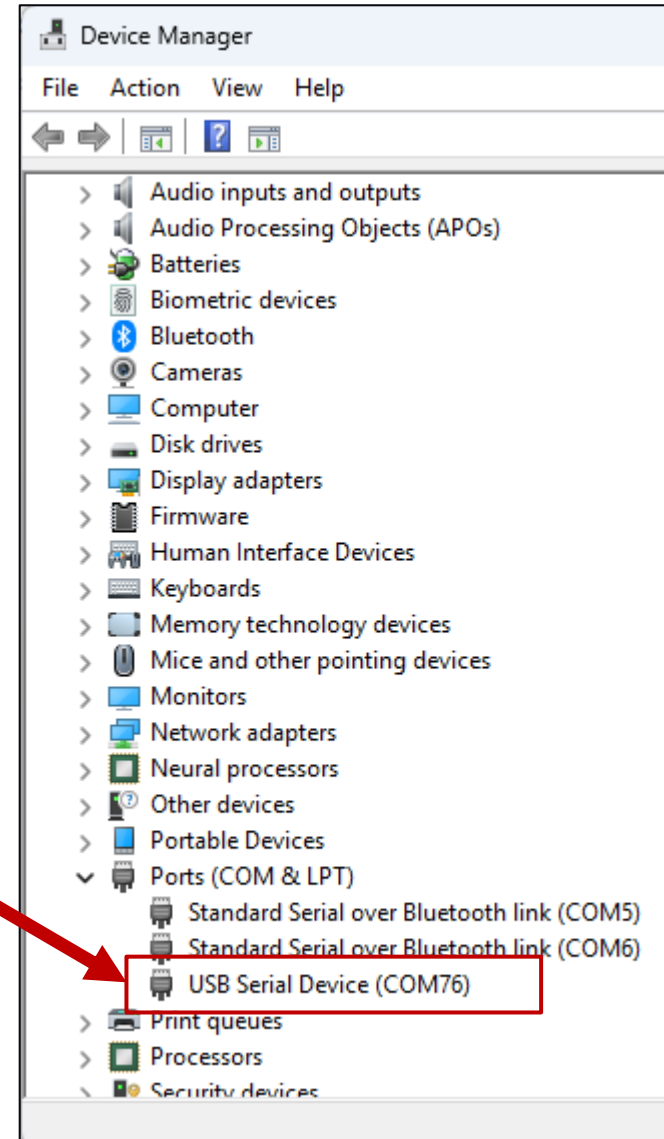
3. Click *Start Monitoring*



Which COM Port Do I Use?

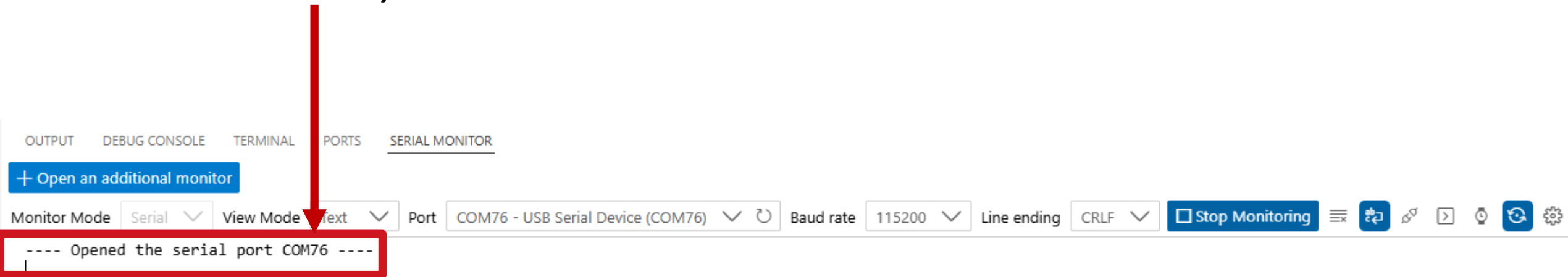
Device Manager

- Click Windows **START** button
 - Type *Device Manager*
- Under: **Ports (COM & LPT)**
 - Look for *USB Serial Device*
- This will show your COM port number – ***Your number is likely to be different!***



Serial Monitor Opened

- You will see:
 - ---- Opened the Serial Port COMxx ----
 - Where: xx is your Serial Port Number



Can I Run Single Lines Of Code?

Like the Python Interactive Interpreter?



- **Yes!**
- It's called **REPL**
 - Read, Evaluate, Print, Loop
- **How do we enter the REPL?**
 - In Serial Terminal send *CTRL+C*
 - This stops executing code!

```
Traceback (most recent call last):
  File "code.py", line 100, in <module>
KeyboardInterrupt:

Code done running.

Press any key to enter the REPL. Use CTRL-D to reload.

Type in a message to send to the serial port.
```

- **Group Task: Send *CTRL+C* to your board**

Enter the REPL

- Click in the message field:

```
Traceback (most recent call last):  
  File "code.py", line 100, in <module>  
KeyboardInterrupt:  
  
Code done running.  
  
Press any key to enter the REPL. Use CTRL-D to reload.  
Type in a message to send to the serial port.
```



- Then press ***Enter***
 - You will then see this prompt:

```
Adafruit CircuitPython 10.1.0-alpha.0-2-g706c4b2abf on 2025-10-27; Microchip Curiosity CircuitPython with same51j20  
>>> |
```

- Shows which version of CircuitPython, the board and MCU we are using
- Group Task: Enter the REPL**

How Do I Get The Object Names?

From the REPL

- In the REPL:
 - Type: `import board`
 - `dir(board)`

```
---- Sent utf8 encoded message: "import board\r\n" ----
```

```
import board
```

```
>>>
```

```
---- Sent utf8 encoded message: "dir(board)\r\n" ----
```

```
dir(board)
```

```
['__class__', '__name__', 'A0', 'A1', 'A2', 'A3', 'A4', 'A5', 'BLE_CLR', 'BLE_RX', 'BLE_TX', 'CAN_RX', 'CAN_STDBY', 'CAN_TX', 'CAP1',  
, 'CS', 'D0', 'D1', 'D10', 'D11', 'D13', 'D2', 'D3', 'D4', 'D5', 'D6', 'D7', 'D8', 'D9', 'DAC', 'DEBUG_RX', 'DEBUG_TX', 'I2C', 'IMU_  
ADDR', 'IMU_INT', 'LCD_BL', 'LCD_CS', 'LCD_MOSI', 'LCD_SCK', 'LCD_SPI', 'LED', 'MISO', 'MOSI', 'NEOPIXEL', 'SCK', 'SCL', 'SDA', 'SD_  
CS', 'SD_MISO', 'SD_MOSI', 'SD_SCK', 'SD_SPI', 'SPI', 'UART', 'VREF', '__dict__', 'board_id']  
>>> |
```

- Notice that one of the objects is named: **LED**
- **Group Task: Get the object names of your board**

How Do I Exit The REPL?

Send CTRL-D to reload your program

- Click on dropdown next to “Send CTRL + C”
- Select “Send CTRL + D”

Press any key to enter the REPL. Use CTRL-D to reload.
---- Sent utf8 encoded message: "\r\n" ----

Adafruit CircuitPython 10.1.0-alpha.0-2-g706c4b2abf on 2025-10-27; Microchip Curiosity CircuitPython with same51j20
>>> |

Type in a message to send to the serial port.

Send as Text

Send Ctrl + C

Send Ctrl + C

Send Ctrl + D

Send Ctrl + X

Send Ctrl + Z

Send Ctrl + C



- In the Serial Terminal you will see
- “Code done running.”

soft reboot

Auto-reload is on. Simply save files over USB to run them or enter REPL to disable.
code.py output:

Hello World!

Code done running.

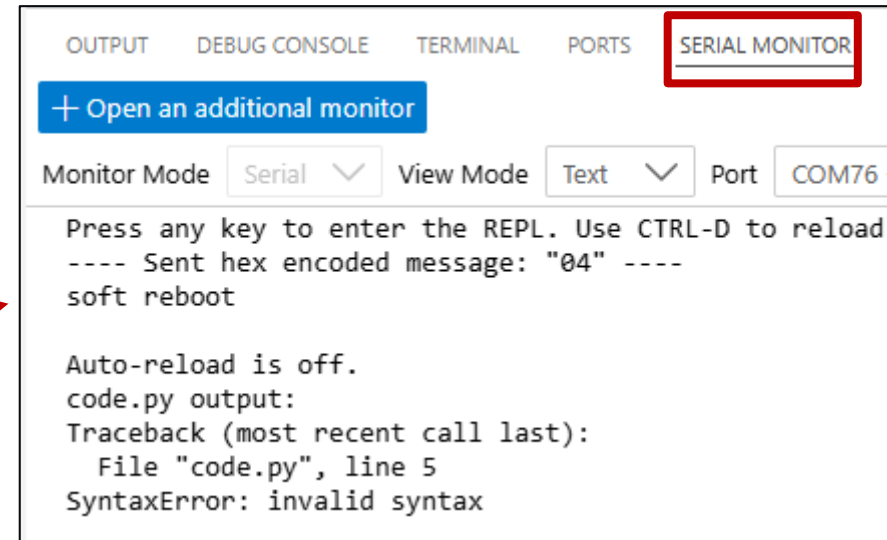
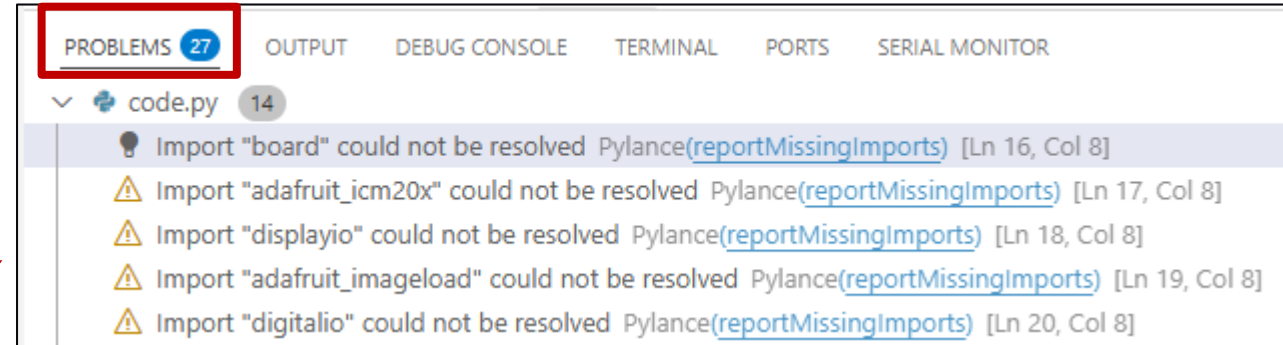
Press any key to enter the REPL. Use CTRL-D to reload.

- This means the code has now completed execution!

IMPORTANT NOTE ABOUT ERRORS!

Use “Serial Monitor” – Not “Problems”

- The extension **CircuitPython** v2 does **NOT YET** support this board.
- **Problems** tab shows “problems” due to this extension not yet supporting this board
- Use **Serial Monitor** tab to view code errors



Getting Started

The README has what you need!

Check the README
Directory Layout

Directory Layout

All modules live flat in `/API` on the CIRCUITPY drive (Adafruit libraries stay in `/lib`):

```
1  CIRCUITPY/  
2  | code.py  
3  | lib/                                ← Adafruit / third-party libraries  
4  |   | asyncio/  
5  |   | adafruit_st7789.mpy  
6  |   | ...  
7  | API/                                ← PyKit Ruler modules  
8  |   | digital_io.py                  ← Dev board modules  
9  |   | analog_io.py  
10 |   | pwm_out.py  
11 |   | cap_touch.py  
12 |   | servo_control.py  
13 |   | uart_comms.py  
14 |   | i2c_bus.py  
15 |   | spi_bus.py  
16 |   | hid_input.py  
17 |   | cpu_temp.py  
18 |   | ble_uart.py  
19 |   | can_bus.py  
20 |   | neopixels.py                  ← Ruler baseboard modules  
21 |   | lcd_display.py  
22 |   | imu_sensor.py  
23 |   | audio_out.py  
24 |   | sd_card.py  
25 |   | bme680.py                    ← I2C breakout modules (QWIIC)  
26 |   | apds9960.py  
27 |   | async_tasks.py                ← Utility modules
```

Getting Started – Before Building a Project

The README has what you need!

Check the
README Module
Quick Reference

Module Quick Reference		
Dev Board Modules		
Module	Class(es)	What it does
digital_io	DigitalOutput, DigitalInput, EdgeDetector	Read buttons/switches; drive LEDs and relays; detect press/release edges
analog_io	AnalogInput, AnalogOutput	Read voltages from sensors (A0–A5); output DC voltage from DAC (board.DAC only)
pwm_out	PWMOutput	Variable duty-cycle signal; LED dimming; buzzer tones; motor speed control
cap_touch	CapTouch	Capacitive touch detect/release on board.A5 (CAP1)
servo_control	ServoController	Position standard RC servo 0°–180°; sweep animations
uart_comms	UARTComms	Send/receive strings over hardware UART (DEBUG or any UART)
i2c_bus	I2CBus	Scan I2C bus; raw register reads/writes; returns bus object for Adafruit drivers
hid_input	HIDKeyboard, HIDMouse, JoystickMouse	USB HID keyboard typing and key combos; mouse movement and clicks; joystick → mouse
cpu_temp	CPUTemperature	On-chip temperature in °C and °F; threshold checks; formatted logging strings
ble_uart	BLEUart	Reset RNBD451 BLE module; send/receive strings wirelessly; connection status tracking
spi_bus	SPIBus	General-purpose SPI transactions with automatic CS and bus locking
can_bus	CANBus	Send and receive CAN frames at 250 kbps; bus state monitoring

Getting Started – Before Building a Project

The README has what you need!

What Inputs and Outputs does your project need?

Choosing Modules for Your Project

Think through what your project needs to sense, process, and output:

INPUTS

digital_io	+ buttons
analog_io	+ sensors
cap_touch	+ touch pad
imu_sensor	+ motion/tilt
bme680	+ temp/humidity
apds9960	+ proximity
apds9960	+ gesture
apds9960	+ color
i2c_bus	+ I2C devices
spi_bus	+ SPI devices
uart_comms	+ serial devices
can_bus	+ CAN network

OUTPUTS

digital_io	+ LEDs, relays
pwm_out	+ motors, buzzers
servo_control	+ servo position
neopixels	+ RGB feedback
lcd_display	+ graphics
audio_out	+ sound / music
ble_uart	+ wireless data
sd_card	+ data logging
hid_input	+ PC automation

Getting Started – Minimal Examples

The README has what you need!

The README has
Minimal examples
to get up and running
quickly!

Minimal Example — Blink the onboard LED

```
1  import pykit_explorer
2  from digital_io import DigitalOutput
3
4  led = DigitalOutput(board.LED)
5
6  while True:
7      led.on()
8      time.sleep(0.5)
9      led.off()
10     time.sleep(0.5)
```

What's Actually Happening Here?

Is it Magic?

- Adafruit Libraries are in CIRCUITPY/lib folder
- APIs are in CIRCUITPY/API folder
- The APIs are just a wrapper for Adafruit Libraries
 - Simplifies building applications!
- **Import pykit_explorer**
 - Pulls in basic libraries and API



What's Actually Happening Here?

No Magic, Just Code!



```
1 import pykit_explorer
2 from digital_io import DigitalOutput
3
4 led = DigitalOutput(board.LED)
5
6 while True:
7     led.on()
8     time.sleep(0.5)
9     led.off()
10    time.sleep(0.5)
```

import pykit_explorer

- Is a hardware initialization shim
- Imports core CircuitPython modules (board, time, built-ins)
- Adds /API to the module search path
- Injects time and board into builtins so all subsequent scripts can use them without explicit imports

What's Actually Happening Here?

No Magic, Just Code!

```
1  import pykit_explorer
2  from digital_io import DigitalOutput
3
4  led = DigitalOutput(board.LED)
5
6  while True:
7      led.on()
8      time.sleep(0.5)
9      led.off()
10     time.sleep(0.5)
```

`from digital_io import
DigitalOutput`

Everyone open the **digital_io.py** file in
CIRCUITPY/API

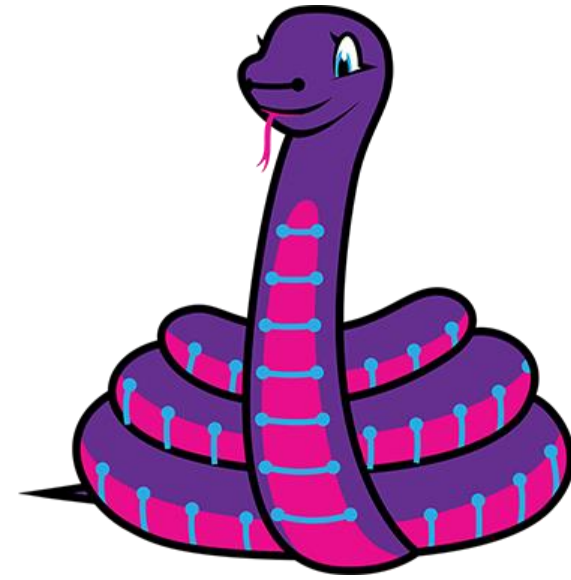
API >  digital_io.py > ..

```
1  """
2  digital_io.py – Digital Input / Output
3  =====
4  Board: Dev Board (any digital-capable pin)
5
6  Provides simple wrappers for reading digital inputs and driving digital outputs.
7  Use this module any time your project needs to:
8      - Read a button, switch, or logic-level signal
9      - Drive an LED, relay, transistor, or other digital load
10     - Detect edges (press / release events)
11
12  Typical pins: D0–D7, D13 (LED), A0–A5 (also usable as digital I/O)
13  """
```

What Methods Are Available to Us?

What can we do with this?

- After the constructor we have:
 - `value` – overloaded, can be getter or setter
 - `On`, `Off`, `Toggle`
 - `Blink` - On time, Off time, Count
- The constructor points to the underlying CircuitPython library *digitalio*



```
def __init__(self, pin):  
    self._pin = digitalio.DigitalInOut(pin)  
    self._pin.direction = digitalio.Direction.OUTPUT  
    self._state = False
```


So Then Where's the Underlying C/C++ Code?

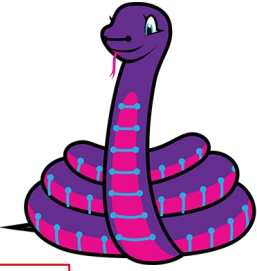
digitalio is a Core CircuitPython Library

[circuitpython](#) / [ports](#) / [atmel-samd](#) / [common-hal](#) / [digitalio](#) / [DigitalInOut.c](#)



From led.on() to a Pin going HIGH

From Python to Hardware in Five Steps!



- Every led.on() call passes through all five layers in microseconds
- **Layer 1 - Your Python code**
The only layer you need to write
- **Layer 2 - API wrapper (digital_io.py)**
Translates on() into a property set
Lives in CIRCUITPY/API on your board
- **Layer 3 - CircuitPython C core**
Adafruit open-source C implementation
Maps Python objects to hardware registers
<https://github.com/adafruit/circuitpython>
- **Layer 4 - ARM Cortex-M4 assembly**
What the CPU actually executes
LDR/STR = Load/Store register
- **Layer 5 - Hardware (SAME51J20A)**
PA14 = the LED pin on PyKit Explorer
An MCU internal transistor switches -> PA14 = 3.3V

```
# Layer 1: Your code (CircuitPython)
led.on()

# Layer 2: API wrapper (digital_io.py)
def on(self):
    self._pin.value = True

# Layer 3: CircuitPython core (C)
// shared-bindings/digitalio
mp_obj_t common_hal_set_value(pin, value) {
    PORT->Group[group].OUTSET.reg
        = pin_mask;
}

# Layer 4: ARM Cortex-M4 assembly
LDR    R0, =0x41008018
MOV    R1, #0x00004000
STR    R1, [R0]

# Layer 5: Hardware (SAME51J20A)
// PA14 pin voltage -> 3.3V (HIGH)
```


SAME51J20A: Registers and Assembly

led.on() = one STR instruction to one address

- **SAME51J20A pin package facts:**
64-pin TQFP/QFN package
2 PORT groups: A and B only
(128-pin parts add groups C and D)
- **OUTSET / OUTCLR are write-only registers**
Write a 1 to a bit -> only that pin changes
Other pins completely unaffected
Safer than read-modify-write on OUT
- **LDR (Load Register)**
Load a memory address into CPU register
- **MOV (Move)**
Load immediate value into register
- **STR (Store Register)**
Write register value to memory address
- **GPIO registers are memory-mapped:**
Writing to 0x41008018 IS the register
No special I/O instructions needed
- **3 instructions -> PA14 changes state**

```
# SAME51J20A: 2 PORT groups (A and B)
PORT_GroupA = 0x41008000 # base addr
PORT_GroupB = 0x41008080 # base + 0x80

# OUTSET: write 1 to set pin HIGH
# OUTCLR: write 1 to set pin LOW
OUTSET_offset = 0x18
OUTCLR_offset = 0x14

# board.LED = PA14 on Curiosity Nano
PA14_mask = (1 << 14) # = 0x00004000

# ARM Cortex-M4: led.on() in assembly
LDR R0, =0x41008018 ; OUTSET addr
MOV R1, #0x00004000 ; PA14 bitmask
STR R1, [R0] ; set PA14 HIGH

# ARM Cortex-M4: led.off() in assembly
LDR R0, =0x41008014 ; OUTCLR addr
MOV R1, #0x00004000 ; PA14 bitmask
STR R1, [R0] ; set PA14 LOW
```

Project 1

Blink the Onboard LED

Copy + Paste Examples: Blink the Onboard LED

README: Minimal Example #1

- This is the embedded systems 'Hello World'
Every embedded systems project starts here
- **import pykit_explorer**
 - Pulls in libraries: time, board, sys, builtins
 - Allows use of modules in /API folder
- **board.LED is the red LED near the USB port**
Not the NeoPixel!
- **while True: runs forever**
Without it, the code runs once and stops
NeoPixel flashes GREEN once when code finishes
- **Try: change the sleep values - what happens if both are 0.1? What if on=0.1, off=1.0?**

Minimal Example — Blink the onboard LED

```
1  import pykit_explorer
2  from digital_io import DigitalOutput
3
4  led = DigitalOutput(board.LED)
5
6  while True:
7      led.on()
8      time.sleep(0.5)
9      led.off()
10     time.sleep(0.5)
```

Copy + Paste Examples: DigitalOutput API

The API has three ways to blink!

- **All three approaches produce a blinking LED**
Each teaches something different:
- **on()/off() - explicit state control**
Good for understanding what the hardware is doing at each line of code
- **toggle() - flip whatever state it is in**
Useful when you don't know current state
Fewer lines, same result
- **blink(on_time, off_time, count)**
Blocking - code waits until done
Great for status flashes
- **Extension: flash SOS in Morse code**
3 short, 3 long, 3 short blinks

```
# Manual on/off with sleep
led.on()
time.sleep(0.5)
led.off()
time.sleep(0.5)
```

1

```
# Toggle flips the current state
while True:
    led.toggle()
    time.sleep(0.5)
```

2

```
# blink() does it all in one call
led.blink(on_time=0.2,
          off_time=0.8,
          count=5)
```

3

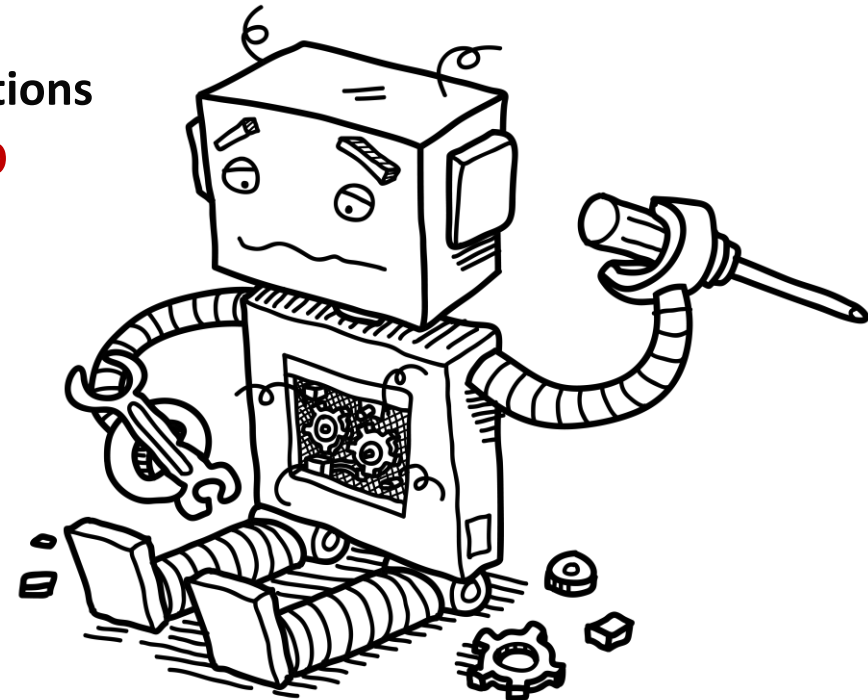
Troubleshooting!

Read Only File System Error When Saving

- **If student cannot save file and board gives Read Only File System error:**
 - First: Try Ejecting the drive, then press Reset button, then try saving file again
 - If that fails:
 - Filesystem may be corrupted!
 - Most likely source:
 - Board has been reset/disconnected while performing write operations
 - **Before starting solution – save files from CIRCUITPY to host computer!!**
- **Solution:**
 - Enter REPL
 - Type:

```
import storage
storage.erase_filesystem()
```

Then copy saved files back to D:\CIRCUITPY



Troubleshooting!

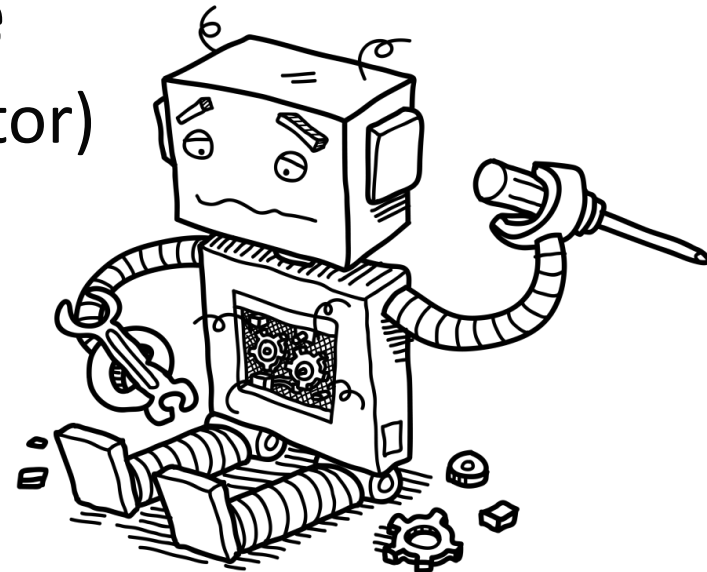
My CIRCUITPY drive disappeared!

If the user was previously successfully writing to D:\CIRCUITPY
But now the drive has disappeared:

1. Check you can access the CPYNANOBOOT drive

1. Double click the RESET button (near the USB connector)
 - You should see the Red LED fading in and out
 - In Windows Explorer you should see D:\CPYNANOBOOT
 - This means the board is likely OK!

(continued on next slide)



Troubleshooting!

CIRCUITPY drive disappeared!

2. Enter *Safe Mode*:

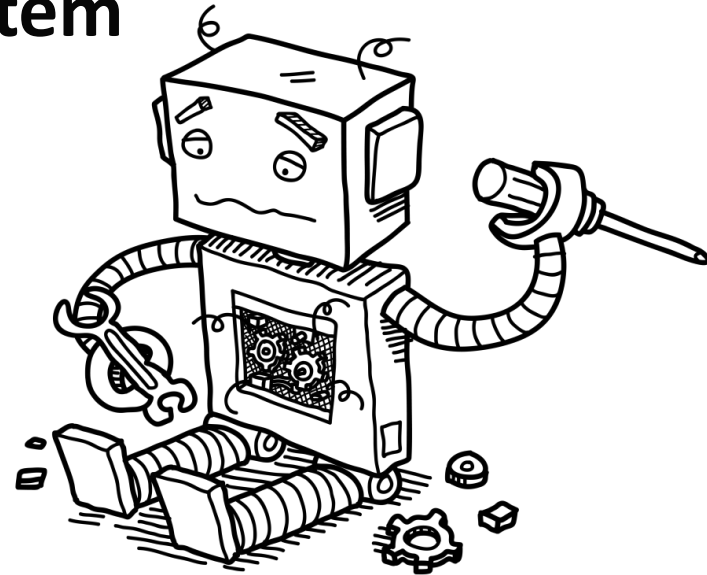
- Press RESET button on Curiosity CircuitPython board
- Immediately you will see 4 Yellow flashes in one second
 - During this time press the RESET button again
 - You will then see 2 yellow flashes and the CIRCUITPY drive will appear!

3. Now you can enter the REPL and erase the filesystem

- Enter REPL, then at prompt `>>>`
- Type:

```
import storage
storage.erase_filesystem()
```

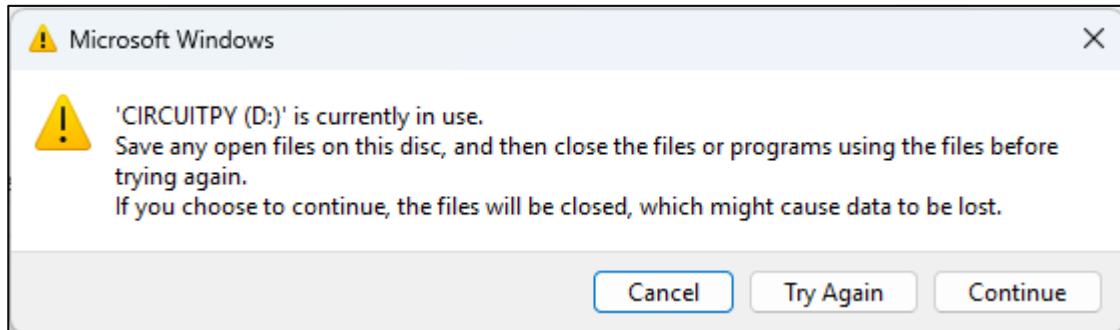
CIRCUITPY drive should reappear!



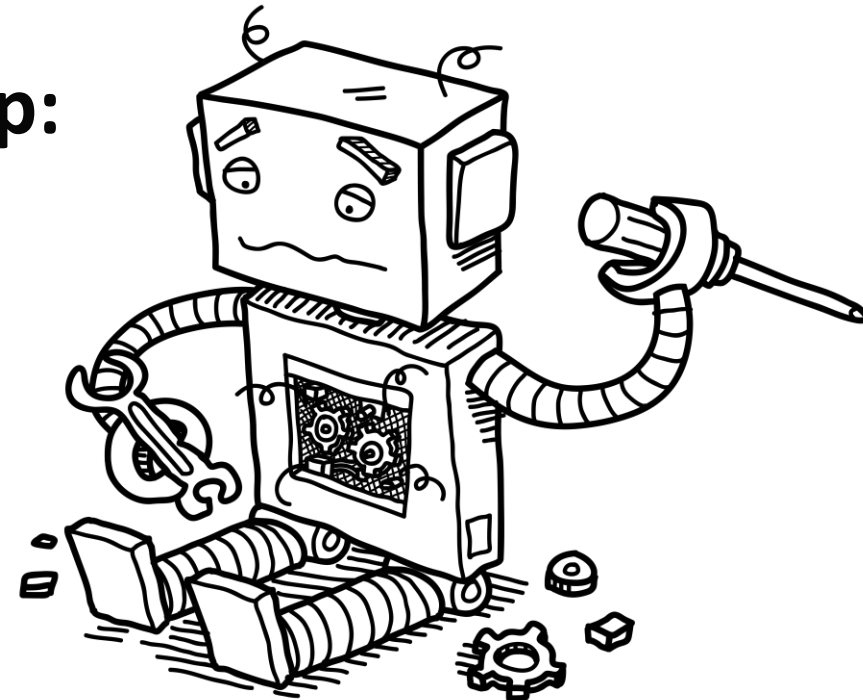
How To Eject The CIRCUITPY Drive

Always Eject before disconnecting cable or pressing reset button

- Before Ejecting D:\CIRCUITPY
 - **Close VS Code first!**
- Failure to close VS Code will give this popup:



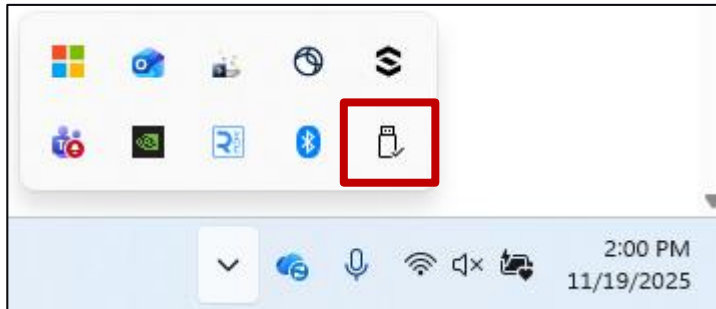
- Click *Cancel*, then close VS Code



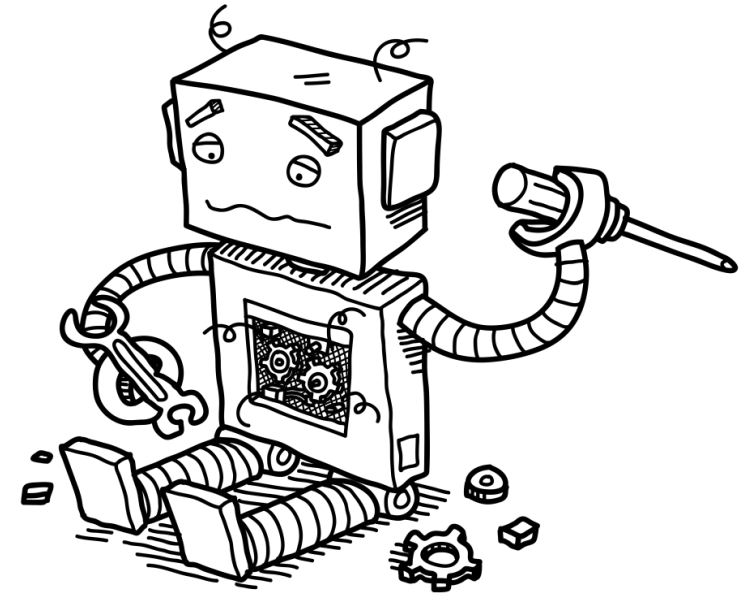
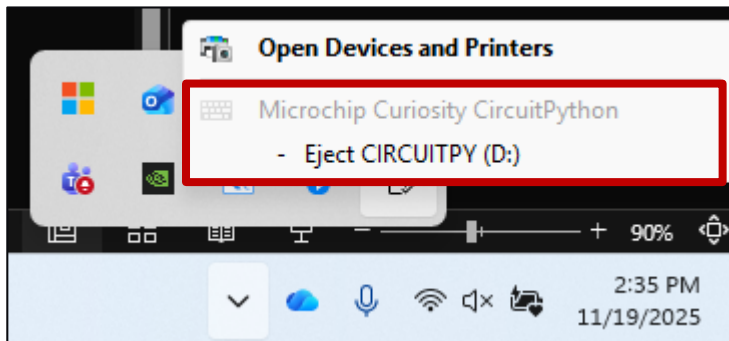
How To Eject The CIRCUITPY Drive

Always Eject before disconnecting cable or pressing reset button

- Click ***Safely Remove Hardware and Eject Media*** button
 - Bottom right corner of Windows

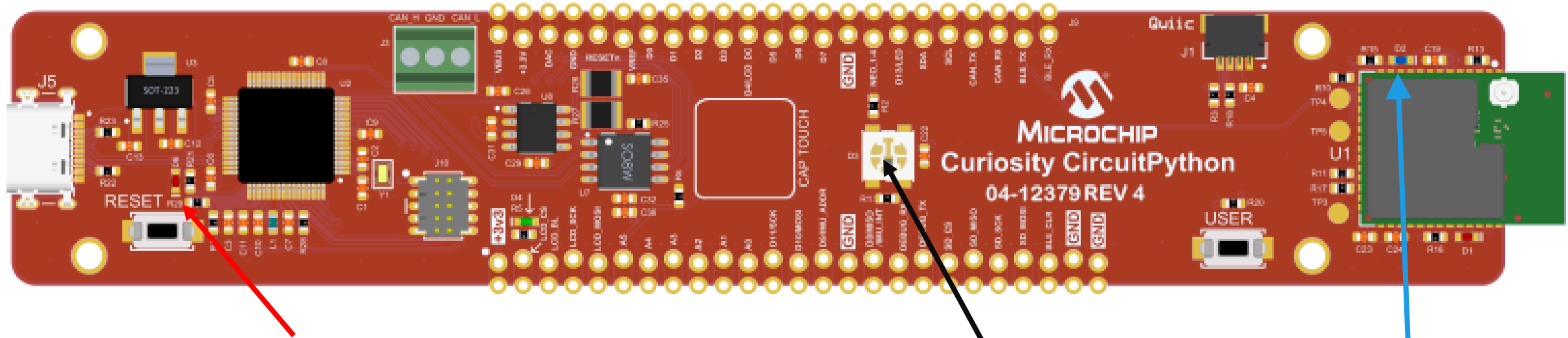


- Click ***Eject CIRCUITPY (D:)***



Where Is The LED?

I can't find it!



- There is a Red LED near the USB connector
- Do not confuse the LED and NeoPixel. The NeoPixel is next to the Cap Touch pad
- The Bluetooth LE module has a Blue LED that blinks periodically (Advertising Mode)

Project 2

Read the User Button

Copy + Paste Examples: : DigitalInput API

README: Minimal Example #2

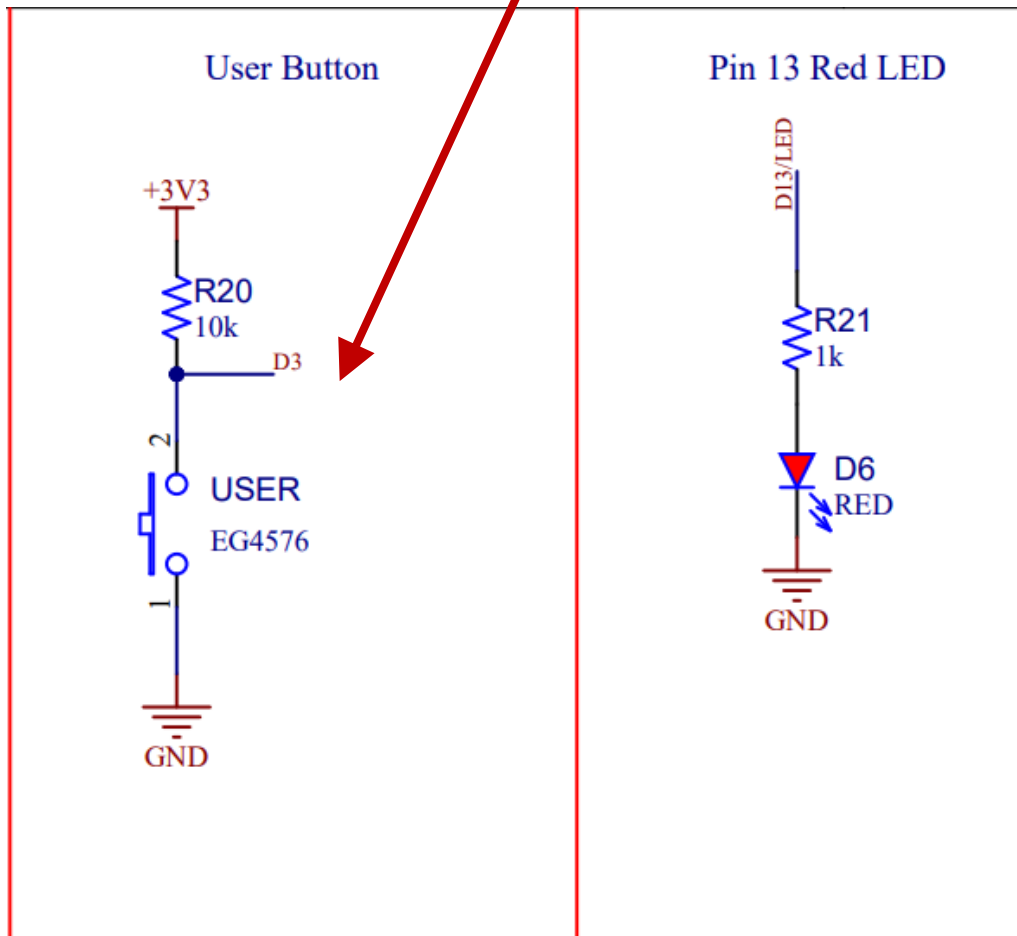
- **btn.value** returns True or False
 - Whether button input pin (D3) is HIGH or LOW
- **btn.is_pressed()** returns True or False
 - Whether button is pressed or not
- **IMPORTANT NOTE:**
 - The return of **value** and **is_pressed** is different!
 - The button on pin D3 is Active-LOW

Minimal Example #2 — Read User Button

```
1  import pykit_explorer
2  from digital_io import DigitalInput
3  btn = DigitalInput(board.D3)
4
5  while True:
6      print(btn.value)           # True = not pressed (active-low)
7      print(btn.is_pressed())   # True when button pulled LOW
8      time.sleep(0.1)           # debounce delay
```

Let's look at the Curiosity CircuitPython schematics!

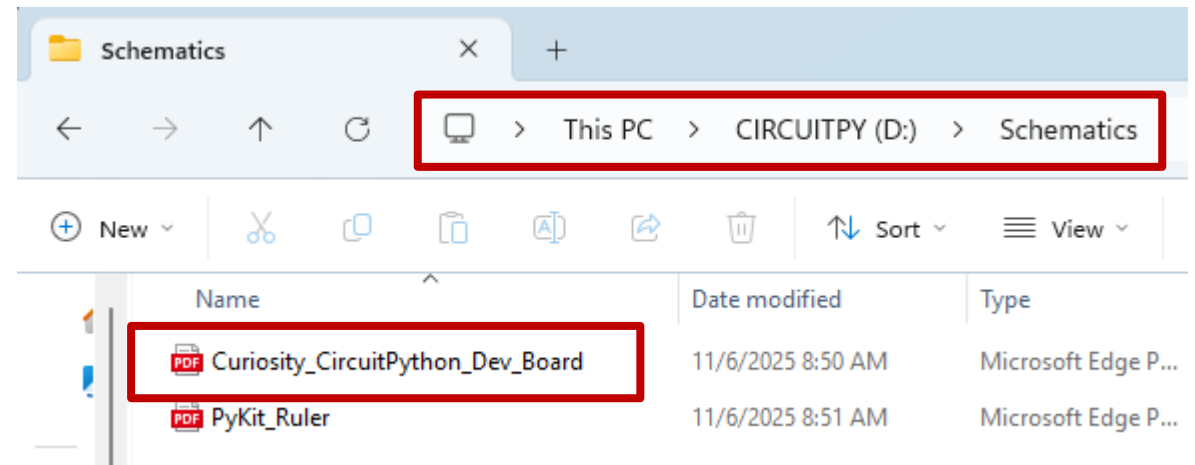
User Button (D3) is Active-LOW!



- Schematics are saved on the board!

- D:\CIRCUITPY\Schematics

Open schematic file in your **browser**:



Copy + Paste Examples: : DigitalInput API

Reading a digital input with DigitalInput

- D3 is the user button on the dev board
From the schematic: button is Active-LOW
Pin reads HIGH when NOT pressed
Pin reads LOW when pressed
- btn.value - raw pin state
True = HIGH (not pressed)
False = LOW (pressed)
- btn.is_pressed() - accounts for wiring
True = button is being pressed
False = button is released
Uses Pull.UP internally -> inverts value
- Watch the Serial Monitor while pressing the button - what do the values show?

Minimal Example #2 — Read User Button

```
1  import pykit_explorer
2  from digital_io import DigitalInput
3
4  btn = DigitalInput(board.D3)
5
6  while True:
7      print(f'Value: {btn.value}')
8      print(f'is pressed: {btn.is_pressed()}')
```

Project 2: Active-LOW and Pull Resistors

Why does btn.value show True when not pressed?

- **Why Active-LOW exists:**
If the wire breaks or disconnects, a pull-up resistor holds the pin HIGH (safe)
Active-LOW = failure means no action taken
- **Used widely in safety-critical systems:**
Emergency stops, low-fuel warnings, boiler water level sensors
- **Pull.UP:** internal resistor connects pin to 3.3V - pin floats HIGH by default
Button connects pin to GND when pressed
- **Pull.DOWN:** opposite - floats LOW, button connects to 3.3V (Active-HIGH)
- **DigitalInput wraps this automatically -**
is_pressed() always returns True on press

```
import digitalio

# DigitalInput default: Pull.UP
btn = DigitalInput(board.D3)
# btn.value = True   (pin pulled HIGH)
# btn.value = False  (button pressed LOW)

# is_pressed() handles the inversion:
# return not self._pin.value
# (because pull is UP -> active-LOW)

# Read the raw value yourself:
import digitalio
pin = digitalio.DigitalInOut(board.D3)
pin.direction = digitalio.Direction.INPUT
pin.pull = digitalio.Pull.UP
print(pin.value)  # True = not pressed
```

Project 3

Toggle LED on Button Press – Putting together Digital Inputs and Outputs

But First - What is an Edge?

The moment a signal changes state - not the state itself

- A digital pin is always either HIGH or LOW

An edge is the moment it transitions

- Falling edge (btn.fell)

HIGH -> LOW transition

Active-LOW button: means **PRESSED**

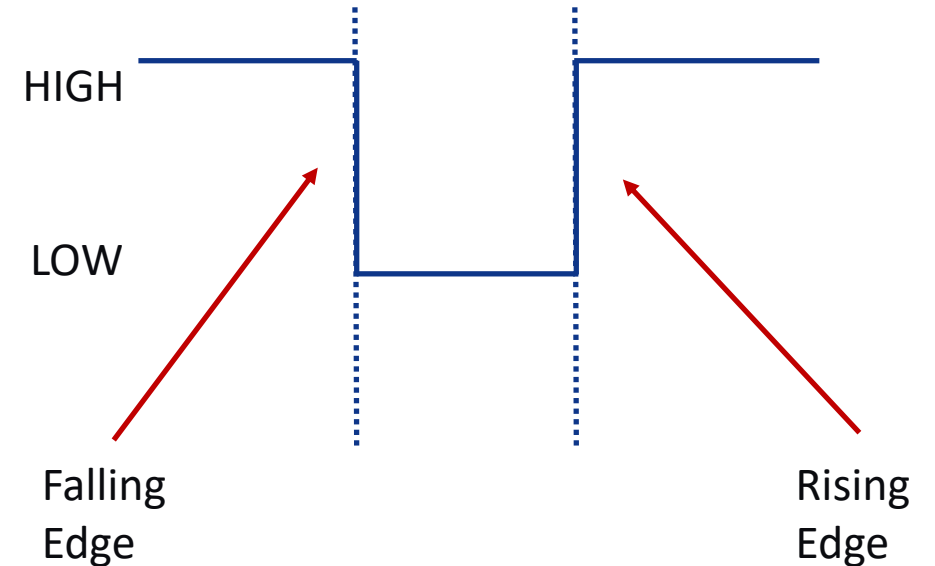
True for exactly ONE loop iteration

- Rising edge (btn.rose)

LOW -> HIGH transition

Active-LOW button: means **RELEASED**

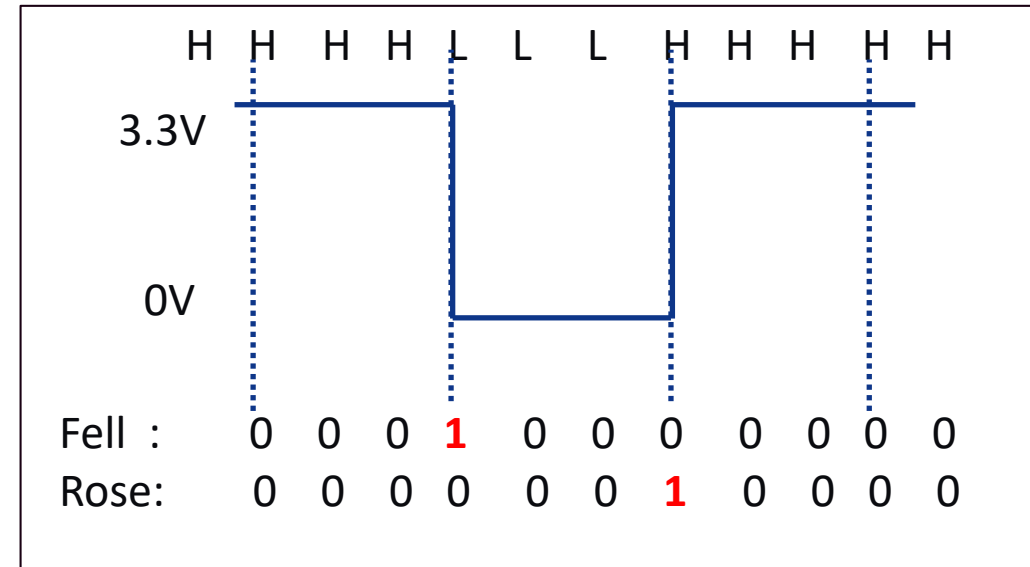
True for exactly **ONE** loop iteration



But First - What is an Edge? (cont.)

The moment a signal changes state - not the state itself

- Why not just use `btn.value`?
`btn.value = H` the whole time not pressed
- The `btn.value` check fires every loop
= 100x per second at 10ms sleep!
- `btn.fell` fires once at the moment
of press -> toggle fires exactly once
- `btn.update()` must be called every loop
It compares current state to last state
to detect the HIGH->LOW transition



H = High
L = Low

Project 3: Toggle LED on Button Press

EdgeDetector fires once per press - perfect for button presses!

Q: What is the EdgeDetector class in the API?

A: Detects the exact moment a pin changes state

btn.fell = True only on HIGH->LOW transition

btn.rose = True only on LOW->HIGH transition

D3 uses a physical Pull-Up resistor (Active-LOW button)

Pin HIGH when not pressed, LOW when pressed

- Try: add led.blink(0.1, 0.1, 3) on press
- Try: count presses, print to Serial Monitor

Minimal Example #3 — Toggle LED on button press

```
1  import pykit_explorer
2  from digital_io import DigitalOutput, EdgeDetector
3
4  led = DigitalOutput(board.LED)
5  btn = EdgeDetector(board.D3)
6
7  while True:
8      btn.update()
9      if btn.fell:
10         led.toggle()
11         time.sleep(0.01)
```

Project 3: DigitalInput and EdgeDetector API

All the methods available to you

- **Key rule:** Call `btn.update()` every single loop
- Flags are only valid for exactly one iteration
- **DigitalInput** (simpler, no edge detection):
- `btn = DigitalInput(board.D3)`
- `btn.is_pressed()` # True while held
- **Extension challenges:**
- **Turn ON on press, OFF on release**
- **Hint:** use `btn.fell` and `btn.rose`
- **Count presses, print count**
- **Hint:** increment a variable on `btn.fell`
- **Flash LED 3x on each press with `blink()`**

```
# DigitalOutput
led = DigitalOutput(board.LED)
led.on()           # Drive HIGH
led.off()          # Drive LOW
led.toggle()       # Flip state
led.blink(0.5,0.5,3) # on,off,count
led.value          # Read current state

# EdgeDetector
btn = EdgeDetector(board.D3)
btn.update()       # MUST call each loop
btn.fell           # True on press (once)
btn.rose           # True on release
btn.value          # Raw pin state
```

Project 4

Control the NeoPixels

Project 4: Control the NeoPixels

Five Individually Addressable RGB LEDs on the PyKit Explorer!

- The Ruler has 5 NeoPixels on board.NEOPIXEL
Default brightness: 0.1 (10%) - safe indoors
- Named colors ready to use:
RED **GREEN** **BLUE** **WHITE**
YELLOW **CYAN** **PURPLE** **OFF**
- Custom color - use an (R,G,B) tuple:
px.fill((128, 0, 255)) # purple
- Animations (all blocking):
color_chase(color) - lights up one by one
rainbow_cycle() - full color wheel
pulse(color) - fade in and out
- Adjust brightness: px.brightness = 0.3

Minimal Example #3 — NeoPixels

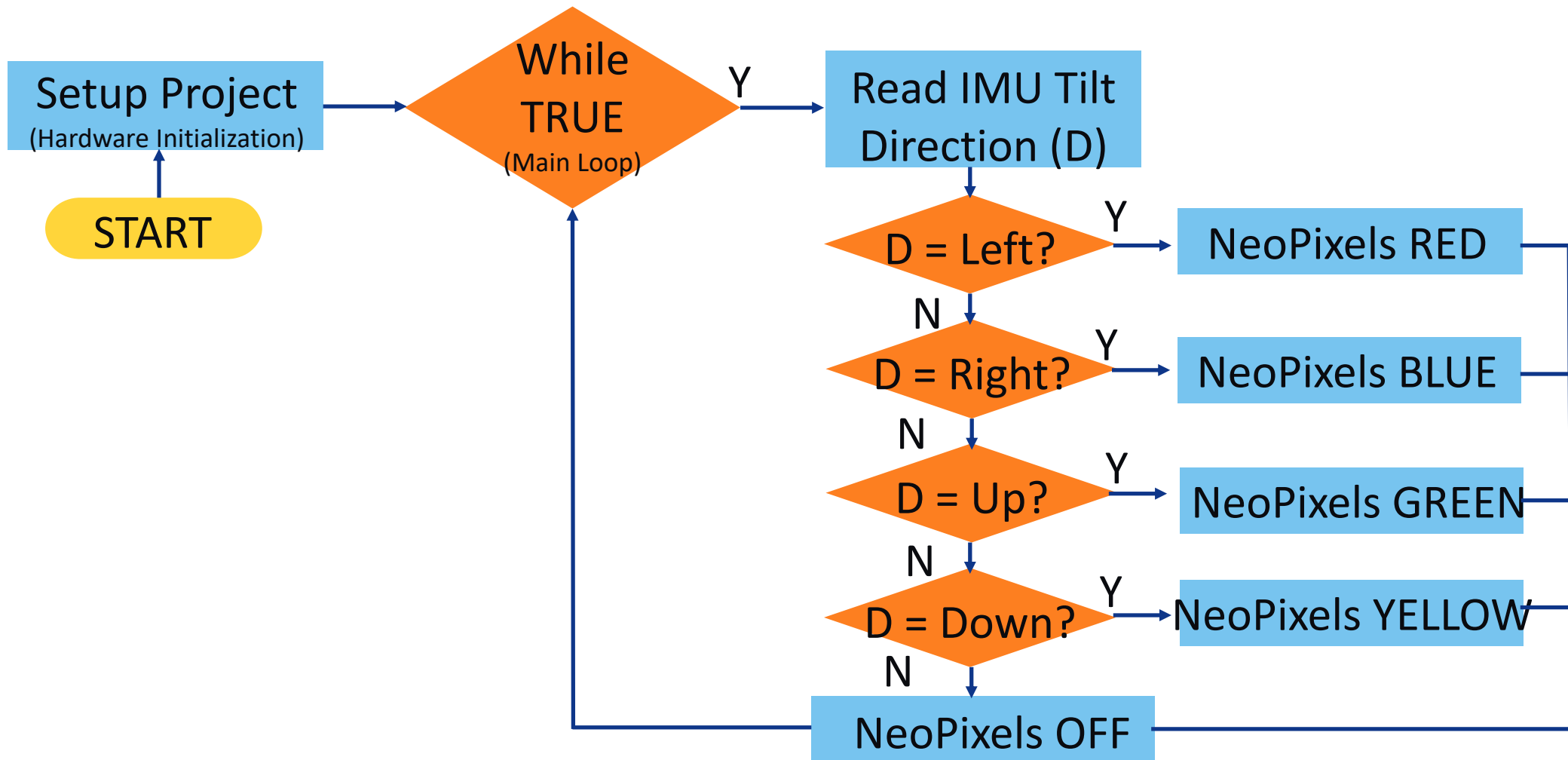
```
1  import pykit_explorer
2  from neopixels import NeoPixels, Colors
3
4  px = NeoPixels() # 5 LEDs, brightness 0.1
5
6  px.fill(Colors.RED)
7  print("All Neopixels should be red")
8  time.sleep(1)
9  px.set(2, Colors.GREEN)
10 print("Pixel 2 should be green")
11 time.sleep(1)
12 px.color_chase(Colors.BLUE)
13 print("Neopixels should chase blue")
14 px.rainbow_cycle(cycles=2)
15 print("Neopixels should cycle through rainbow colors")
16 px.off()
```

Project 5

Read IMU and Control NeoPixels

Project 5-1: Tilt-Controlled NeoPixels

Intro to Flowcharts!



Project 5-1: Tilt-Controlled NeoPixels

IMUSensor reads the ICM20948 on the Ruler

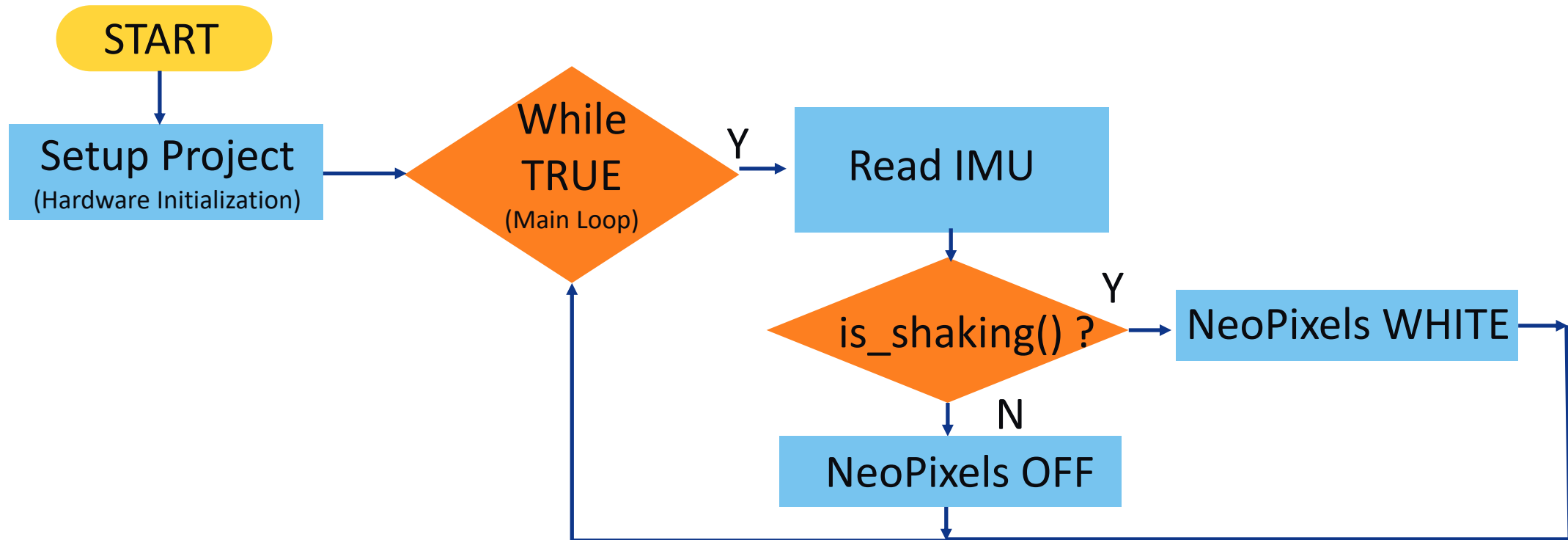
- IMUSensor API wraps the ICM20948 9-axis IMU
Accelerometer, gyroscope, magnetometer
- **tilt_direction()** returns a string:
LEFT RIGHT UP DOWN FLAT
Threshold = 3.0 m/s² (adjustable)
- **Raw data also available:**
ax, ay, az = imu.acceleration # m/s²
gx, gy, gz = imu.gyro # rad/s
imu.tilt_angle_x # degrees
- **First step: print tilt_direction() to Serial Monitor and tilt the board to see the values before adding NeoPixels**

Minimal Example #5 — Tilt-controlled NeoPixel colours

```
1  import pykit_explorer
2  from imu_sensor import IMUSensor
3  from neopixels import NeoPixels, Colors, OFF
4
5  imu = IMUSensor()
6  px = NeoPixels()
7
8  while True:
9      direction = imu.tilt_direction()
10     if direction == "LEFT":
11         px.fill(Colors.RED)
12     elif direction == "RIGHT":
13         px.fill(Colors.BLUE)
14     elif direction == "UP":
15         px.fill(Colors.GREEN)
16     elif direction == "DOWN":
17         px.fill(Colors.YELLOW)
18     else:
19         px.off()
20
```

Project 5-2: IMU Shake Detection

Intro to Flowcharts!



Project 5-2: IMU Shake Detection

is_shaking() and print_all() for debugging

- **print_all()** logs to Serial Monitor:
Accel X/Y/Z in m/s²
Gyro X/Y/Z in rad/s
Mag X/Y/Z in uT
- **is_shaking(threshold=15.0)**
Gravity always contributes ~9.8 m/s²
Default 15.0 catches real shaking

Extension challenges:

- Trigger rainbow_cycle() on shake
- Map tilt_angle_x to map_value() to show tilt amount as a NeoPixel bar graph

Minimal Example — IMU shake detection

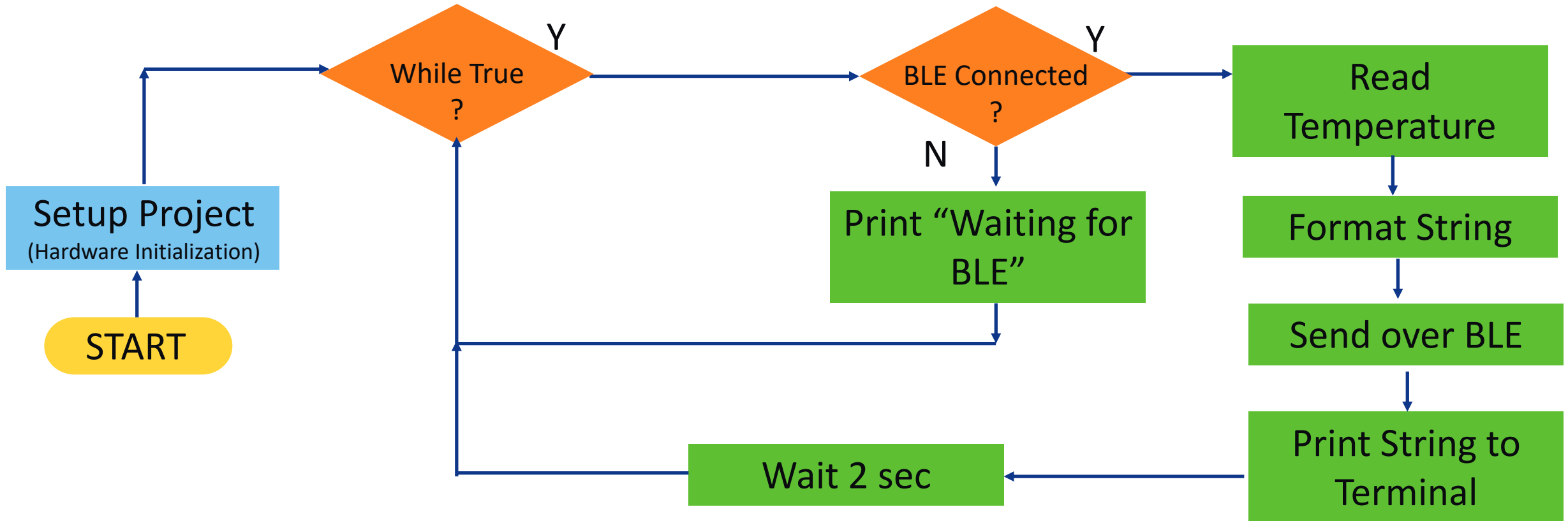
```
1  import pykit_explorer
2  from imu_sensor import IMUSensor
3  from neopixels import NeoPixels, WHITE, OFF
4
5  imu = IMUSensor()
6  px = NeoPixels()
7
8  while True:
9      imu.print_all()          # Serial Monitor
10     if imu.is_shaking():
11         px.fill(WHITE)
12         time.sleep(0.2)
13     else:
14         px.off()
15         time.sleep(0.1)
```

Project 6

BLE Temperature Logger

Project 6: BLE Temperature Logger

Send CPU temperature wirelessly to your phone



Project 6: BLE Temperature Logger

First Install the Microchip Bluetooth Data app!



Android



iPhone



Project 6: BLE Temperature Logger

Send CPU temperature wirelessly to your phone

- Hardware: RNBD451 BLE on dev board
UART: board.BLE_TX / BLE_RX

ble.connected - True when phone connected

ble.send(string) - transmits over BLE

ble.receive() - call every loop!

Strips module status messages

CPUTemperature.formatted_string()

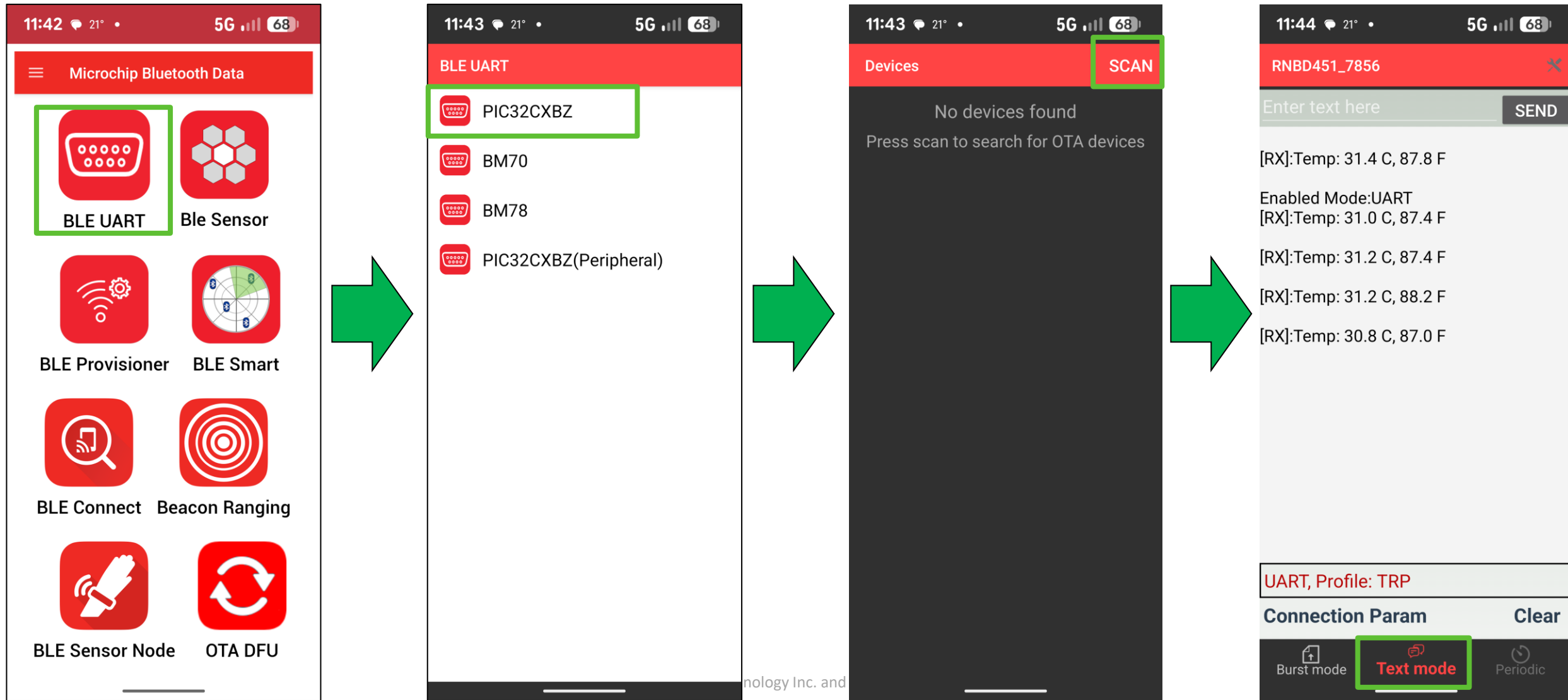
Returns '25.3 C, 77.5 F'

Accuracy: +/- 5 deg C

Minimal Example #6 — BLE temperature logger

```
1  import pykit_explorer
2  from ble_uart import BLEUart
3  from cpu_temp import CPUTemperature
4
5  ble = BLEUart()
6  temp = CPUTemperature()
7
8  while True:
9      ble.poll() # process connection status messages
10     if ble.connected:
11         ble.send(f"Temp: {temp.formatted_string()}\n")
12         print(f"Temp: {temp.formatted_string()}")
13     time.sleep(2)
```

How to Use the App



Project 6: Receiving BLE Commands

Two-way BLE: control NeoPixels from phone

Type commands in the
Microchip Bluetooth Data app

Board receives them and acts on them

- Always call `.strip()` to remove newlines

- Use `repr(cmd)` to debug whitespace

- Extension challenges:

Send custom color commands from phone

Minimal Example #6-2 — Receiving BLE Commands

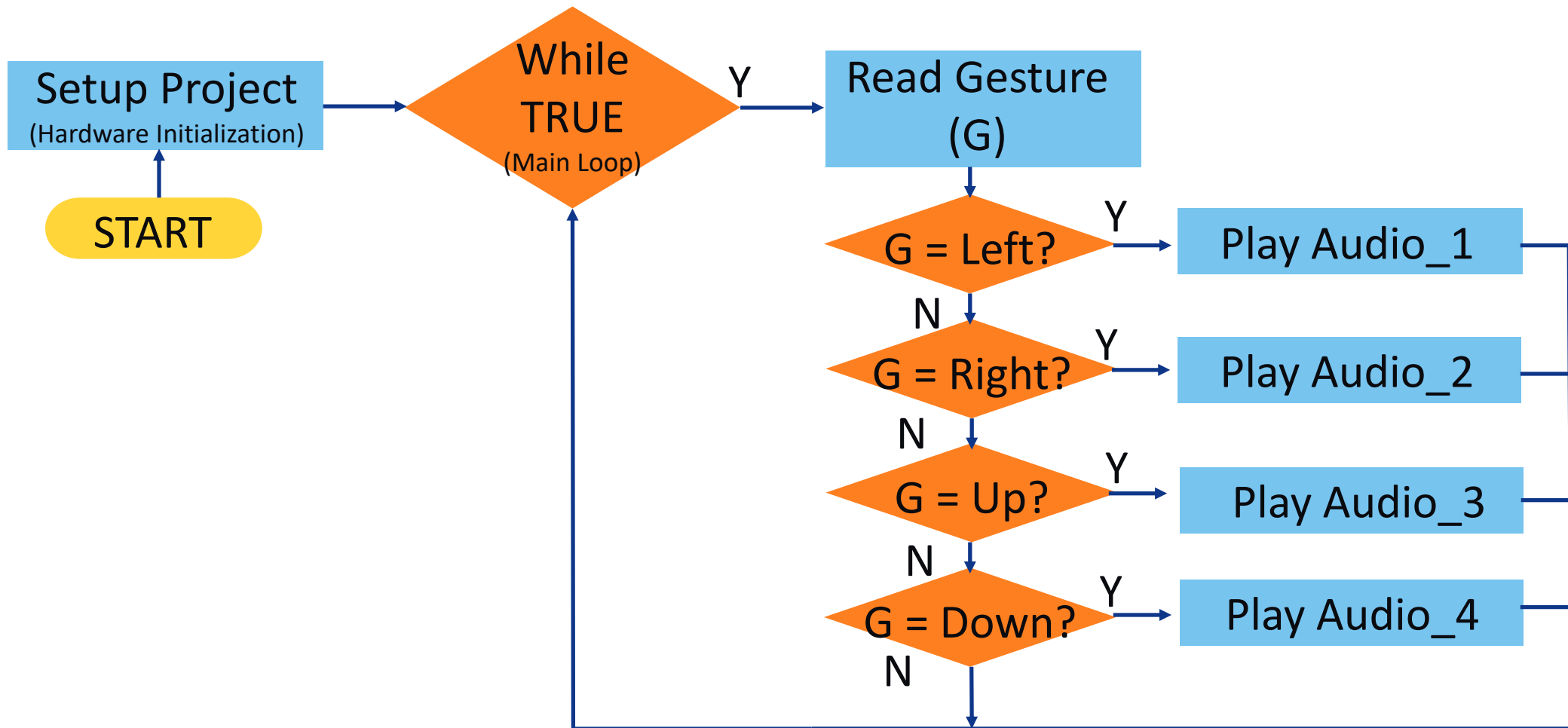
```
1  import pykit_explorer
2  from ble_uart import BLEUART
3  from neopixels import NeoPixels, Colors, OFF
4
5  ble = BLEUART()
6  px = NeoPixels()
7
8  while True:
9      cmd = ble.receive().strip()
10     if cmd == 'RED': px.fill(Colors.RED)
11     elif cmd == 'GREEN': px.fill(Colors.GREEN)
12     elif cmd == 'OFF': px.off()
13     if cmd:
14         print(f'Got: {repr(cmd)}')
15     time.sleep(0.05)
```

Project 7

Gesture-Controlled Audio

Project 7: Gesture-Controlled Audio

APDS9960 swipe detection + WAV playback



Project 7: Gesture-Controlled Audio

APDS9960 swipe detection + WAV playback

Hardware: APDS9960 on QWIIC connector

Pass my_i2c.bus (not my_i2c)

Three mutually exclusive modes:

enable_proximity() - distance 0-255

enable_gesture() - swipe detection

enable_color() - RGBC reading

Only ONE mode active at a time!

- **Gesture constants (from apds9960):**
GESTURE_UP/DOWN/LEFT/RIGHT
- **WAV files: Mono, 16-bit PCM, 22050Hz max**

Minimal Example — APDS9960 gesture → WAV audio

```
1 import pykit_explorer
2 from i2c_bus import I2Cbus
3 from apds9960 import APDS9960Sensor, GESTURE_UP, GESTURE_DOWN, GESTURE_LEFT, GESTURE_RIGHT
4 from audio_out import AudioOutput
5
6 my_i2c = I2Cbus()
7 sensor = APDS9960Sensor(my_i2c.bus)
8 audio = AudioOutput()
9
10 sensor.enable_gesture()
11
12 while True:
13     g = sensor.gesture()
14     if g == GESTURE_UP:
15         audio.play_wav("AudioFiles/304.wav")
16     elif g == GESTURE_DOWN:
17         audio.play_wav("AudioFiles/140.wav")
18     elif g == GESTURE_LEFT:
19         audio.play_wav("AudioFiles/210.wav")
20     elif g == GESTURE_RIGHT:
21         audio.play_wav("AudioFiles/320.wav")
```

Project 7: Gesture Debug and Extensions

Troubleshoot and go further

- **gesture()** returns 0 when nothing detected
Only act on non-zero: if g:
- **gesture_name(g)** returns human string:
'UP' 'DOWN' 'LEFT' 'RIGHT' 'NONE'
- **wait_for_gesture(timeout)**
Blocks until swipe or timeout
Returns GESTURE_NONE on timeout
- **Extension challenges:**
Swipe UP/DOWN to change NeoPixel color
Swipe LEFT/RIGHT to change brightness
Build a gesture theremin:
Use proximity to control pitch
audio.play_tone(freq, duration)
Map proximity value to frequency

```
# Debug: print gesture names to console
sensor.enable_gesture()
while True:
    g = sensor.gesture()
    if g: # non-zero = detected
        print(sensor.gesture_name(g))
```

```
# Proximity mode - distance 0-255
sensor.enable_proximity()
while True:
    dist = sensor.proximity
    print(f'Proximity: {dist}')
    time.sleep(0.1)
```

```
# Block until gesture or timeout
g = sensor.wait_for_gesture(timeout=5.0)
print(sensor.gesture_name(g))
```

Project 8

Color Sensing and Reproduction Using NeoPixels

Project 8: Color Sensing with NeoPixels

Match NeoPixel color to ambient light

- **enable_color()** activates the RGBC sensor
RGBC = Red, Green, Blue, Clear (16-bit)
- **color_as_neopixel()** converts to 8-bit:
Shifts each 16-bit channel right by 8
Returns tuple ready for px.fill()
- **Other color methods:**
sensor.color - raw (r,g,b,c)
sensor.color_as_hex() - 0xRRGGBB int
sensor.print_color() - debug to console
- **Try: Hold colored objects in front of the sensor and watch NeoPixels change**
 - Cellophane works well!
- **Needs decent ambient light to work well**

Minimal Example — APDS9960 RGBC values to NeoPixels

```
1 import pykit_explorer
2 from i2c_bus import I2Cbus
3 from apds9960 import APDS9960Sensor
4 from neopixels import NeoPixels
5
6 my_i2c = I2Cbus()
7 sensor = APDS9960Sensor(my_i2c.bus)
8 px = NeoPixels()
9 sensor.enable_color()
10
11 while True:
12     r,g,b,clear = sensor.color
13     print(f'R:{r} G:{g} B:{b} C:{clear}')
14     neo = sensor.color_as_neopixel()
15     px.fill(neo)
16     time.sleep(0.2)
```

Project 8: Color Sensing with NeoPixels

Inspect and use the raw RGBC data

- **sensor.color** returns 16-bit values (0-65535)
NeoPixel needs 8-bit (0-255)
`color_as_neopixel()` does: `val >> 8`
- **Clear channel = total light intensity**
Can normalise R/G/B against it
Compensates for brightness changes
- **color_as_hex()** returns 0xRRGGBB
Useful for display color properties
- **Extension challenges:**
Sample and store a reference color
Compare live readings to the reference
Light NeoPixels when color matches
Hint: compare channels within tolerance
e.g. `abs(r1-r2) < 20`

Minimal Example — APDS9960 RGBC values to NeoPixels

```
1 import pykit_explorer
2 from i2c_bus import I2CBus
3 from apds9960 import APDS9960Sensor
4 from neopixels import NeoPixels
5
6 my_i2c = I2CBus()
7 sensor = APDS9960Sensor(my_i2c.bus)
8 px = NeoPixels()
9 sensor.enable_color()
10
11 while True:
12     r,g,b,clear = sensor.color
13     print(f'R:{r} G:{g} B:{b} C:{clear}')
14     neo = sensor.color_as_neopixel()
15     px.fill(neo)
16     time.sleep(0.2)
```


Project 9: BME680 Air Quality Display

Show Temperature Range at a Glance with NeoPixels

- Reads the current temperature band (LOW, MED, HIGH, or above) and maps it to a NeoPixel color:
 - **Blue** — low temperature
 - **Green** — medium temperature
 - **Yellow** — high temperature
 - **Red** — above high threshold

Thresholds defined in
`/API/bme680.py`

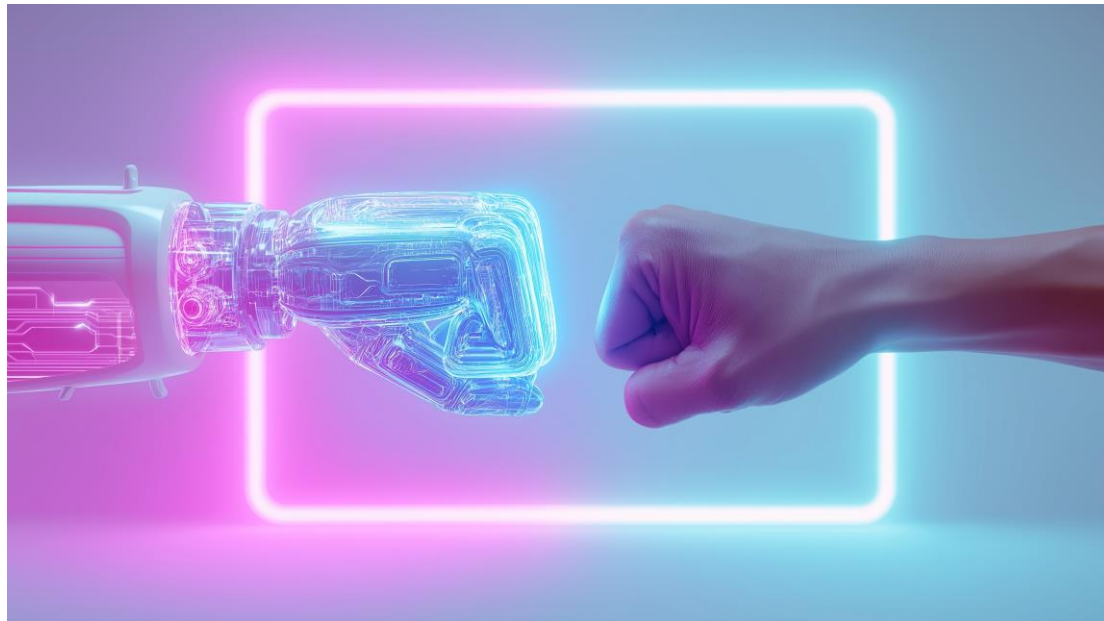
Minimal Example — BME680 air quality display

```
1  import pykit_explorer
2  from i2c_bus import I2CBus
3  from bme680 import BME680Sensor
4  from neopixels import NeoPixels, GREEN, YELLOW, RED, BLUE
5
6  my_i2c = I2CBus()
7  sensor = BME680Sensor(my_i2c.bus, elevation_m=362)
8  px      = NeoPixels()
9
10 while True:
11     sensor.print_all()
12     level = sensor.temperature_level()
13     if level == "LOW":
14         px.fill(BLUE)
15     elif level == "MED":
16         px.fill(GREEN)
17     elif level == "HIGH":
18         px.fill(YELLOW)
19     else:
20         px.fill(RED)
21     time.sleep(1)
```

Complex Project #1

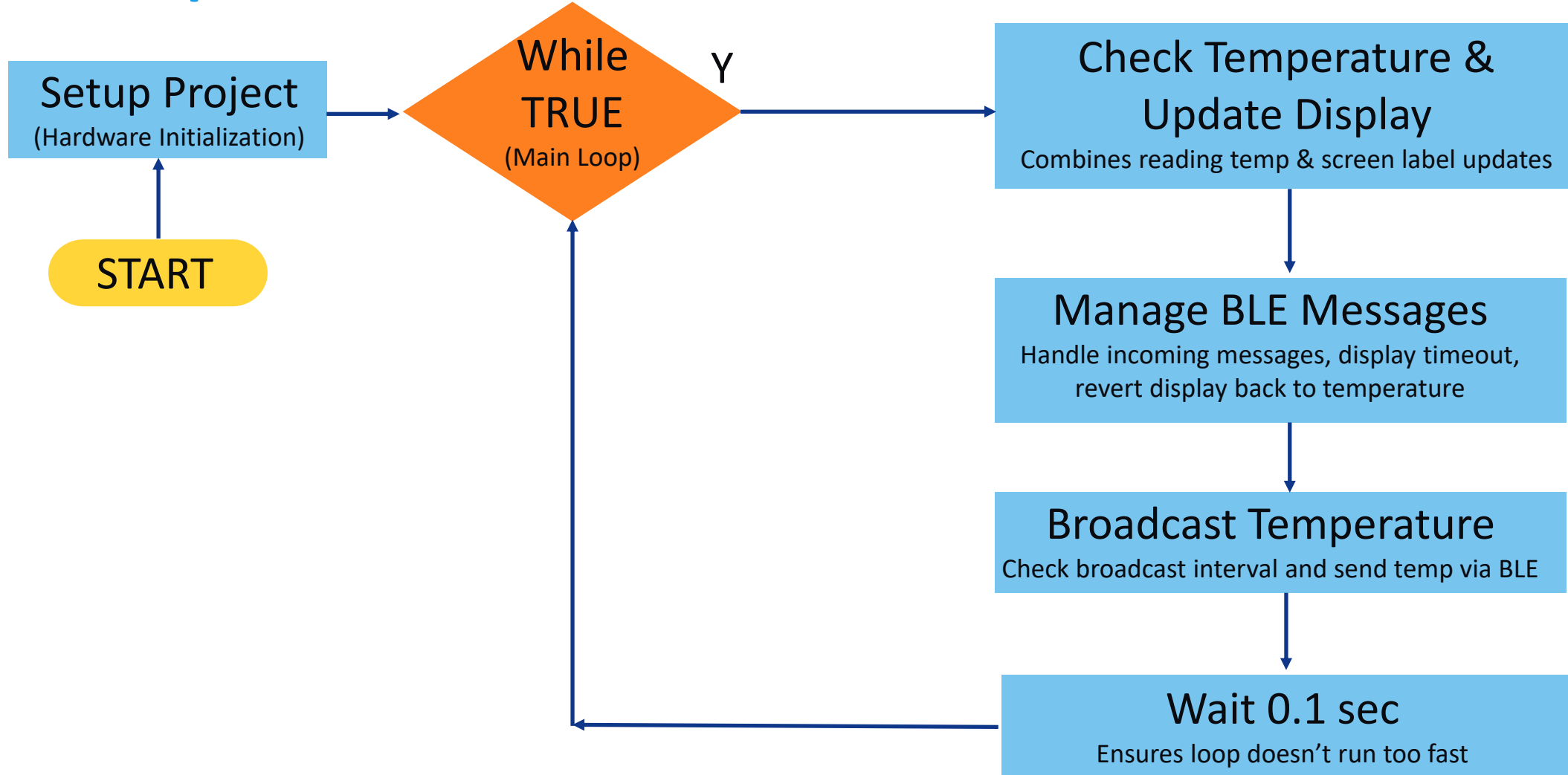
Using BLE, NeoPixels, LCD, CPU Temperature together

- Read the CPU Temperature
- Write it to the LCD
- Change NeoPixel Color based on Temperature thresholds
- Write it out to a connected BLE device (phone)
- Check for incoming BLE messages from phone & display them



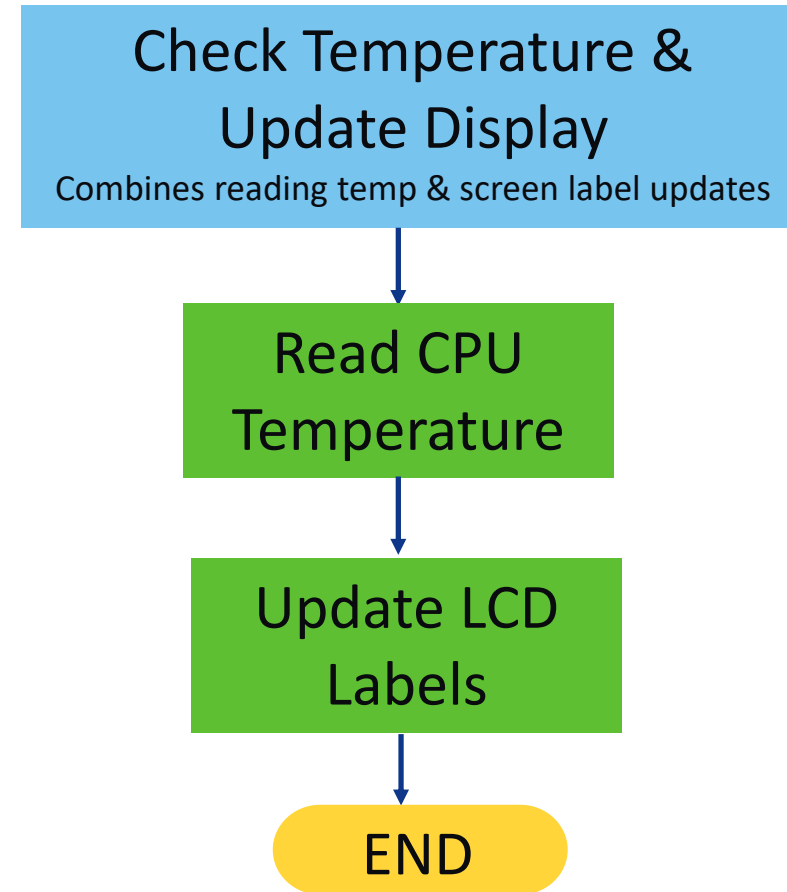
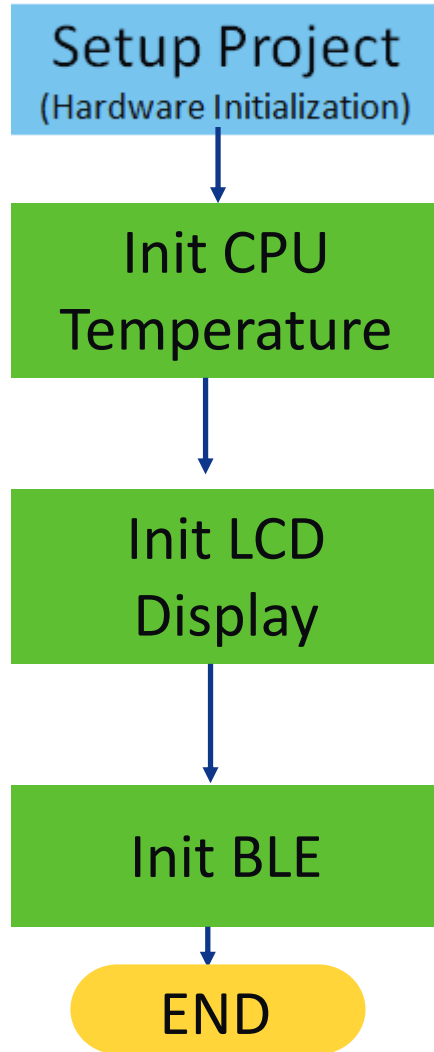
Complex Project #1

More Complex Flowcharts



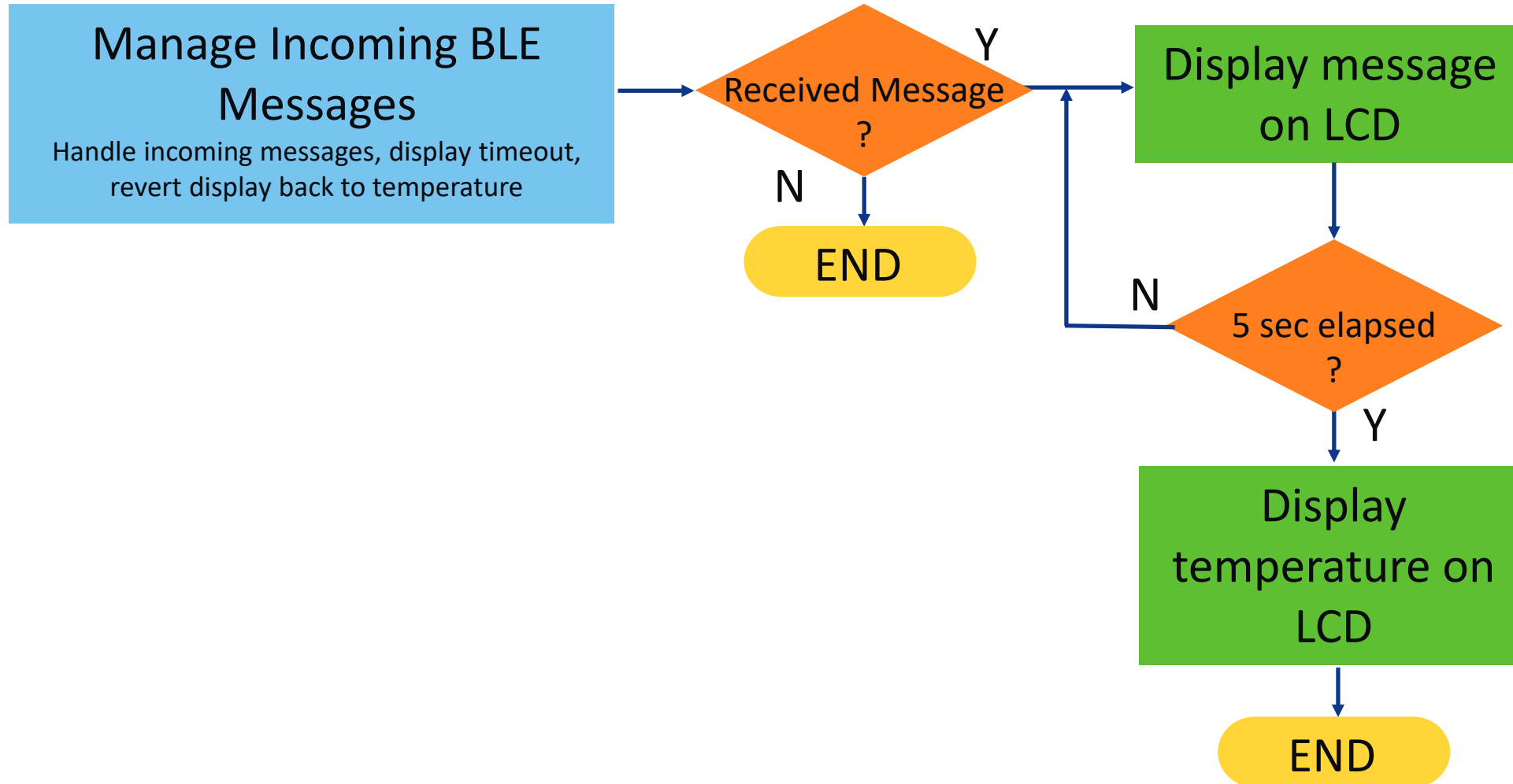
Complex Project #1

Flowchart Subsections



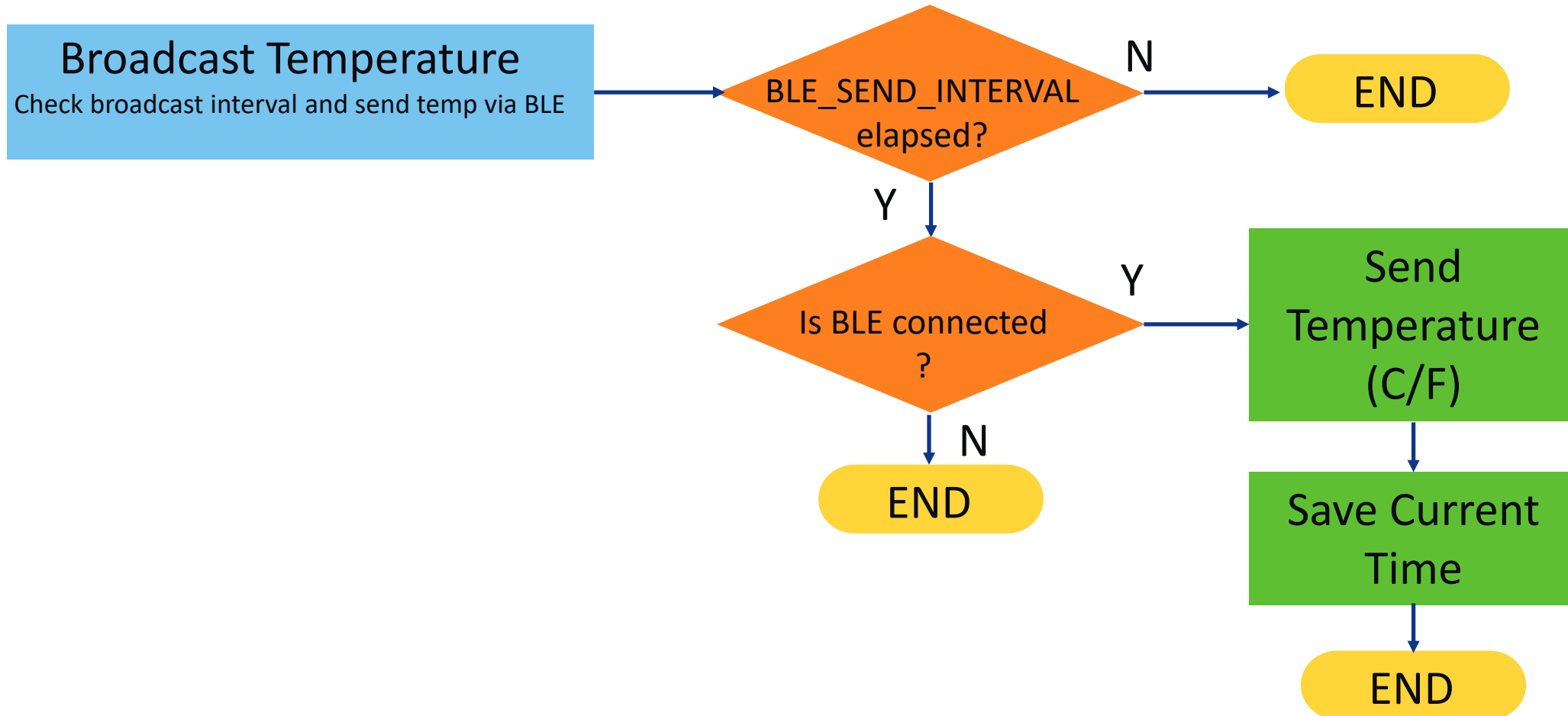
Complex Project #1

Flowchart Subsections



Complex Project #1

Flowchart Subsections



Let's Build A Project Together!

Complex Project #2 – IMU, NeoPixels, LCD, BLE

- **Requirements:**

- Read IMU data
- Make NeoPixels change color using IMU data
- Write IMU data & NeoPixel colors to LCD and BLE

- **Questions:**

- **What modules do we need?**
- **Which classes do we need?**
- **Which methods do we need?**
- **Which IMU data will we use?**
 - Accelerometer, Gyroscope, Magnetometer
- **What datatype/s does:**
 - The IMU return?
 - NeoPixels take?
 - LCD labels take?
 - BLE send take?